

This thesis was submitted to the Chair of Machine Learning and Reasoning.

Open-World Robotic Assembly with Physically Feasible Planning

Bachelor Thesis

Presented by

Wenbo Ma
433351

Supervised by Daniel Swoboda, M.Sc.

1st Examiner Prof. Hector Geffner, Ph.D.

2nd Examiner Prof. Dr. rer. nat. Alexander Ferrein

Aachen, May 25, 2026

Abstract

Autonomous robotic assembly requires the coordinated integration of perception, task planning, and contact-aware manipulation under geometric and physical constraints. This thesis presents a vision-based robotic assembly system for LEGO Duplo construction tasks in semi-structured tabletop environments. Given a structured YAML description of a target structure, the system generates assembly plans whose ordering, grasp access, collision constraints, and press-in stability are checked before execution.

The proposed framework integrates assembly-sequence planning, open-vocabulary perception, and robotic execution within a unified pipeline. Assembly sequences are generated using the Assembly-by-Disassembly (ABD) principle by reversing valid disassembly orders. To improve execution feasibility, the planner incorporates grasp reachability analysis, collision-aware feasibility verification, and press-stability reasoning based on a voxel representation of the target structure.

For perception, RGB-D observations are processed using GroundingDINO, Segment Anything (SAM), and FoundationPose to perform open-vocabulary object detection, instance segmentation, and CAD-based 6D pose estimation. The execution module is implemented using ROS 2 and MoveIt 2 and performs grasping, placement, alignment, and stud-engagement insertion through a collision-aware state-machine controller.

The proposed system is evaluated in a MuJoCo-based LIBERO simulation environment and compared against a vision-language-action (VLA) baseline on long-horizon assembly tasks with increasing structural complexity. Experimental results show that the proposed structured pipeline achieves higher success rates, lower stability violation rates, and more reliable insertion performance, particularly in complex assembly scenarios with constrained insertion operations and dense structural dependencies. The results highlight the importance of explicit geometric reasoning and physically grounded execution for robust robotic assembly systems.

Contents

Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Setting and Challenges	2
1.3 Proposed Approach	4
2 Background	6
2.1 Assembly-Sequence Planning and Symbolic Reasoning	6
2.2 Perception and Part Identification	7
2.2.1 Open-Vocabulary Perception	8
2.2.2 Instance Segmentation	9
2.2.3 6D Pose Estimation	9
2.3 Robotic Execution and State-Machine Control	11
2.4 ROS and MoveIt	12
3 Related Work	14
3.1 Assembly-Sequence Planning	14
3.1.1 Physics-Based Simulation for Assembly	15
3.1.2 Learning-Based Assembly Planning	16
3.2 Perception for Robotic Assembly	17
3.2.1 Classical Vision-Based Assembly Perception	17
3.2.2 Deep Learning-Based Perception	18
3.2.3 6D Pose Estimation	18
3.3 Low-Level Execution and Task Control	19
3.3.1 Classical Robotic Assembly Execution	19
3.3.2 Visual Servoing and Contact Control	20
3.3.3 Vision-Language-Action Models	20
3.4 Integrated Robotic Assembly Pipelines	21
3.4.1 LLM-Based Task Planning	21
3.4.2 Vision-Language Reasoning for Assembly	22
3.4.3 Symbolic-Geometric Hybrid Planning	22
4 Methods	23

4.1	System Overview and Setup	23
4.2	Assembly-Sequence Planning with Physical Feasibility	24
4.3	Perception: Open-Vocabulary Visual Pose Estimation	31
4.4	Execution: Collision-Aware State Machine	33
4.4.1	Overall Execution Pipeline	33
4.4.2	Grasping and Placement Execution	34
4.4.3	Collision-Aware Motion Planning and Velocity Scheduling	35
4.4.4	Failure Handling	36
5	Evaluation	37
5.1	Experimental Setup	37
5.2	Baseline Method	38
5.3	Evaluation Metrics	39
5.3.1	Evaluation Results	39
6	Conclusion	43
	List of Acronyms	45
	List of Symbols	46
	List of Figures	47
	List of Tables	49
	List of Algorithms	50
	List of References	51

1 Introduction

1.1 Motivation

Autonomous robotic assembly is an important research area in robotics and intelligent manufacturing, aiming to enable robots to perceive parts, plan assembly orders, and execute manipulation actions in physically constrained environments. Typical research and application scenarios include furniture assembly, industrial component assembly, tabletop object construction, warehouse packaging, and service-robot manipulation tasks in household environments [15, 24, 40].

These application scenarios require robots not only to manipulate objects, but also to perceive the environment and generate physically executable assembly plans. During the assembly process, the robot must continuously reason about spatial relations, contact interactions, and structural dependencies between different parts. In many tasks, the robot must determine feasible assembly orders, select suitable grasp configurations, align components precisely, avoid collisions with surrounding structures, and maintain stable intermediate assemblies during insertion operations. Even small geometric errors or incorrect assembly decisions may lead to failed insertions or unstable structures. Consequently, robotic assembly is widely regarded as a representative long-horizon manipulation problem that requires the coordinated integration of perception, task planning, motion generation, and manipulation control [15, 40]. Compared with simpler robotic manipulation settings, assembly tasks typically involve stronger geometric constraints, richer contact interactions, and more complex dependencies between sequential manipulation actions.

With recent advances in robotics and Embodied AI, robotic systems are increasingly moving beyond fixed industrial production lines toward more open and dynamic environments [38, 49]. In practical scenarios such as flexible manufacturing, tabletop manipulation, and service robotics, robots must operate under cluttered scenes, uncertain object configurations, and previously unseen objects [4, 15, 21, 27]. These requirements also motivate recent vision-language-action (VLA) systems, which attempt to map visual observations and language instructions directly to robot actions [5, 7, 8, 19]. However, contact-rich assembly still requires strong geometric consistency, precise insertion, and stable intermediate structures. As a result, modern robotic assembly systems increasingly require the integration of autonomous planning, environment perception, collision-aware manipulation, and constrained motion generation under kinematic and geometric constraints.

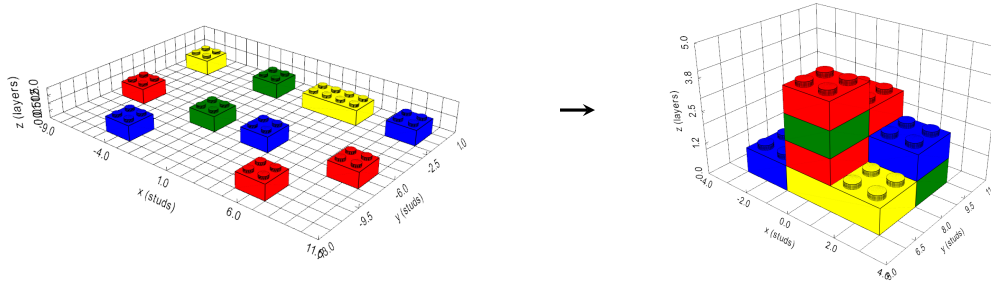


Figure 1.1: Example robotic assembly task used in this work. Separated LEGO Duplo bricks are initially placed in the pick area (left), and the robot constructs the target structure in the assembly area (right).

1.2 Problem Setting and Challenges

LEGO-style assembly has become a widely used platform in robotic assembly research because it provides a compact and reproducible environment while still preserving many fundamental challenges of real-world assembly tasks [3, 32]. The discrete stud-grid structure simplifies geometric representation, and the modularity of LEGO components enables task generation with varying structural complexity. In this work, LEGO Duplo bricks are used as the assembly object category because their size is compatible with parallel-jaw robotic grasping while still requiring precise alignment and press-in insertion. At the same time, the sequential and interdependent nature of LEGO construction introduces substantial planning complexity. Since each assembly action may constrain future grasp accessibility, insertion directions, and structural support conditions, robotic systems must determine physically feasible assembly orders before execution.

Assembly Sequence Planning (ASP) is a computationally difficult combinatorial search problem [36, 46]. As the number of assembly components increases, the search space of feasible assembly orders grows rapidly, making exhaustive forward search increasingly impractical. Moreover, robotic assembly planning requires not only determining a valid assembly order, but also verifying geometric and physical feasibility during execution, including grasp reachability, collision-free insertion, and intermediate-structure stability. In contact-rich assembly tasks such as LEGO insertion, the robot must continuously reason about insertion directions, neighboring obstacles, structural dependencies, and support conditions.

Besides high-level planning, robotic assembly also depends heavily on robust perception capabilities. In long-horizon assembly tasks, the spatial configuration of the workspace continuously changes as new parts are inserted into the structure. Partial occlusions, self-occlusions, varying

object orientations, and dense object arrangements further increase the difficulty of accurate object localization and pose estimation. Accurate 6D pose estimation is particularly important in robotic assembly because even small translational or rotational errors may propagate into insertion failures during contact-rich manipulation [43, 45, 48].

Open-world tabletop settings further require perception modules that can generalize beyond fixed object categories and fixed scene templates. For assembly tasks, this motivates the use of open-vocabulary perception together with instance-level segmentation and CAD-based pose estimation, so that task-level part descriptions can be grounded into metric object poses for downstream planning and manipulation.

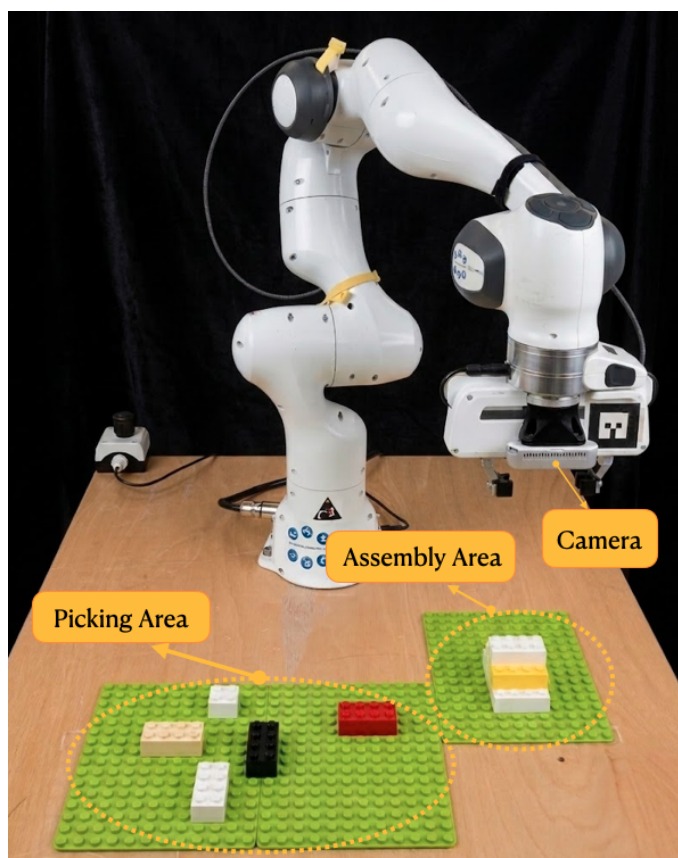


Figure 1.2: Robotic assembly workspace used in this work, consisting of a Franka Emika Panda robot arm, an RGB-D camera, a pick area containing separated LEGO Duplo bricks, and an assembly area for target structure construction.

Although recent end-to-end visuomotor policies and VLM/VLA-based systems have demonstrated promising generalization capabilities [5, 7, 8, 19], purely implicit reasoning may still struggle to maintain reliable geometric consistency and physical feasibility in contact-rich assembly tasks. Therefore, combining symbolic assembly reasoning with explicit geometric verification provides a practical compromise between generalization capability and physical reliability. This motivates the development of structured robotic assembly pipelines that combine

high-level assembly reasoning with explicit geometric verification and physically grounded execution modules.

These challenges become particularly significant in LEGO-style assembly. Successful LEGO insertion requires precise alignment between studs and receiving holes while simultaneously maintaining collision-free motion and stable intermediate structures. During execution, the robot must continuously reason about insertion direction, neighboring obstacles, grasp accessibility, and structural support conditions. Small pose estimation errors may accumulate across multiple assembly stages and eventually lead to insertion failure or structural instability. Consequently, explicit geometric reasoning and physically grounded planning remain important for robust robotic assembly systems.

Motivated by the RoboCup Smart Manufacturing League (SML) assembly workbench setting, this thesis studies a LEGO Duplo robotic assembly task under semi-structured tabletop environments with randomized initial object configurations. Figure 1.2 illustrates the robotic assembly workspace used in this work. The setup consists of a Franka Emika Panda robot arm equipped with a parallel-jaw gripper and an RGB-D camera mounted above the workspace. The workspace is divided into a pick area containing initially separated LEGO Duplo bricks and an assembly area where the target structure is constructed.

As illustrated in Figure 1.1, the robot is required to construct a target structure from initially separated parts. The task requires end-to-end robotic autonomy, including generating executable assembly sequences, identifying and localizing target parts, selecting collision-free grasp configurations, planning feasible trajectories, and executing accurate placement and stud-engagement insertion operations.

1.3 Proposed Approach

The proposed system follows a structured planning–perception–execution pipeline. The target structure is represented using a structured YAML specification describing part identities and assembly relations. Based on this representation, the system derives executable assembly sequences using Assembly-by-Disassembly reasoning together with grasp reachability analysis and press-stability verification. RGB-D observations are processed using GroundingDINO, Segment Anything (SAM), and FoundationPose to perform open-vocabulary object detection, instance segmentation, and CAD-based 6D pose estimation [21, 27, 45]. The estimated object poses are subsequently used by a collision-aware execution module integrated with ROS 2, MoveIt 2, and OMPL-based motion planning [10, 11] to perform grasping, alignment, insertion, and press-in assembly operations.

The main contributions of this work are summarized as follows:

- We propose a structured robotic assembly framework for LEGO Duplo assembly tasks that integrates Assembly-by-Disassembly planning, open-vocabulary perception, and collision-aware robotic execution within a unified pipeline. The proposed framework further combines voxel-based grasp reachability analysis, press-stability verification, and collision-aware motion planning to improve the physical feasibility and execution robustness of long-horizon robotic assembly tasks.
- We conduct a systematic evaluation in a MuJoCo-based LIBERO simulation environment and compare the proposed system against a VLM/VLA-based baseline. Experimental results demonstrate that explicit geometric reasoning and structured assembly planning significantly improve execution robustness, insertion stability, and manipulation efficiency in contact-rich robotic assembly scenarios.

2 Background

Robotic assembly can be viewed as a structured process that bridges high-level task planning and low-level physical execution. To understand how the proposed system operates, this section reviews the foundational concepts involving assembly sequence planning, perception, and grasp feasibility, followed by the computational tools used in this work.

2.1 Assembly-Sequence Planning and Symbolic Reasoning

Assembly Sequence Planning (ASP) is one of the central problems in robotic assembly and intelligent manufacturing [36, 46], since the order in which parts are assembled directly affects both feasibility and execution efficiency. In robotic assembly tasks, the assembly sequence not only determines precedence relationships between parts, but also influences whether subsequent operations can be successfully executed under kinematic reachability, collision avoidance, and grasp/placement constraints. Many assembly tasks exhibit strong structural and geometric dependencies, where certain components must first provide supporting structures before later parts can be correctly inserted or stably placed. Therefore, an effective assembly plan must ensure not only logical consistency, but also physical realizability at every intermediate assembly state.

Traditional assembly planning methods commonly employ symbolic representations to describe part states and assembly relationships. In symbolic planning, the environment is abstracted into discrete symbolic states and symbolic actions that transition between states. Each action is typically associated with preconditions and effects, describing when the action can be executed and how the world state changes after execution. For example, in the classical BlocksWorld formulation [12], the assembly process is represented as transitions between discrete block configurations, while grasping and placement operations modify the symbolic structure state. Such approaches are effective for modeling task dependencies and logical ordering, and have therefore been widely adopted in task planning and robotic reasoning research.

However, purely symbolic planning methods often ignore physical constraints present in real robotic systems. An assembly action that appears valid at the symbolic level may still fail during execution due to collisions, insufficient grasping clearance, restricted insertion directions, or structural instability. Consequently, assembly sequences generated purely through symbolic reasoning are not always executable in practical robotic environments. To improve physical feasibility, modern robotic assembly systems typically incorporate geometric feasibility, kinematic constraints, and structural stability analysis directly into the planning process [40].

One widely used assembly planning strategy is Assembly-by-Disassembly (ABD), which derives an assembly order by first reasoning about how a completed structure can be disassembled and subsequently reversing the resulting disassembly sequence to obtain an executable assembly sequence [30, 46]. Compared with forward assembly search from the initial state, ABD naturally reduces the search space because, at any given stage, only parts that are currently removable are considered. Parts blocked by neighboring structures, unreachable for grasping, or critical for maintaining structural stability are automatically excluded from the candidate set.

A key advantage of ABD is that removability implicitly satisfies many geometric and structural feasibility constraints. If a part can be removed from the current structure without collision, the reversed operation usually corresponds to a feasible insertion path during assembly. As a result, reversing a valid disassembly sequence often produces an assembly order that respects precedence relations while avoiding obviously infeasible transitions.

Beyond symbolic ordering, assembly planning must additionally consider physical feasibility constraints during execution. Even if a sequence is logically valid, execution may still fail when a part cannot be grasped, oriented, or inserted as required. Geometric constraints include collision avoidance, sufficient gripper clearance, and the existence of feasible insertion or approach directions. Stability constraints ensure that intermediate structures remain physically supported throughout the assembly process.

A typical example occurs in LEGO-style assembly, where parts are connected through downward press-in motions to engage studs and complete the connection. A brick may appear valid in the symbolic assembly order, yet the required pressing operation can destabilize or topple the partially assembled structure. Therefore, when evaluating whether a part can be selected as the next disassembly candidate, the planner must additionally check press stability during the reversed assembly process. Consequently, ABD provides a natural framework for integrating geometric feasibility, grasp reachability, insertion constraints, and structural stability into assembly planning, and has therefore been widely adopted in robotic assembly research [3, 26, 40].

2.2 Perception and Part Identification

Vision-based perception is a fundamental component of automated robotic assembly, since robots are required to detect, localize, and reason about parts under varying degrees of uncertainty. Unlike traditional industrial assembly settings, where object positions and categories are fixed and known in advance, assembly tasks in open or semi-structured environments are characterized by unknown initial configurations, cluttered scenes, and partial occlusions. Consequently, robotic systems must continuously perceive and interpret the workspace state to provide reliable environmental information for downstream grasping, placement, and motion planning.

In robotic assembly tasks, the perception module typically consists of three major components: open-vocabulary perception, instance segmentation, and 6D pose estimation. Together, these components determine how effectively the robot can understand the scene and directly influence the performance of downstream planning and execution modules.

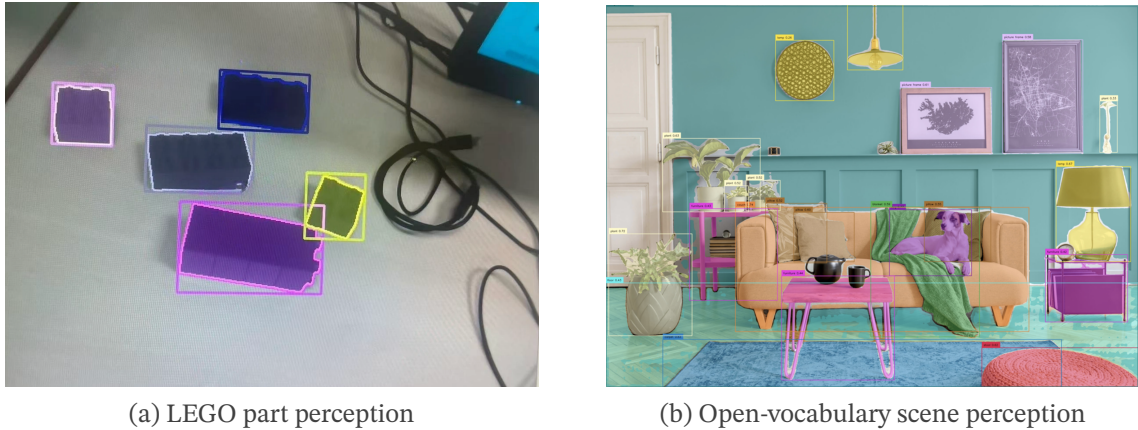


Figure 2.1: Examples of open-vocabulary perception in cluttered environments. The right image is adapted from [35].

2.2.1 Open-Vocabulary Perception

Open-vocabulary perception aims to detect and identify objects using semantic descriptions instead of relying on a fixed set of predefined categories. Traditional object detection systems are usually trained on closed-category datasets, where the model can only recognize object classes that appear during training. This assumption is often too restrictive for robotic assembly tasks, where target parts may be specified by task-level descriptions, such as color, shape, or part type, and where new object categories may appear in different tasks.

Figure 2.1 illustrates this difference between task-specific part perception and more flexible open-vocabulary scene grounding. Recent vision-language detection models address the closed-category limitation by aligning visual regions with natural language descriptions [27, 33]. Instead of predicting only fixed class labels, these models take text prompts as input and localize image regions that correspond to the given semantic query. This makes it possible to search for objects such as “red 2x4 brick” or “blue cube” without training a dedicated detector for every possible part category.

GroundingDINO extends transformer-based object detection with language-guided grounding [27]. It combines image features and text features through cross-modal fusion, allowing the detector to associate words in a prompt with relevant image regions. The output of the model is a set of bounding boxes together with confidence scores for the queried objects. In robotic assembly, such text-conditioned detection is useful because part specifications can be derived directly from task descriptions or structured product files.

In this work, open-vocabulary perception provides the first stage of the perception pipeline. It identifies candidate object regions based on semantic part descriptions, which are then refined by instance segmentation and used for downstream 6D pose estimation. This allows the perception system to remain flexible with respect to new part descriptions while still providing spatial information required for grasping and assembly execution.

2.2.2 Instance Segmentation

While object detection provides coarse object localization, robotic assembly tasks often require more precise geometric information for grasp synthesis, contact reasoning, placement verification, and downstream pose estimation. Consequently, instance segmentation plays an important role in robotic assembly perception systems by providing pixel-level object masks and accurate object boundaries.

Instance segmentation is particularly important in robotic assembly because assembly operations frequently involve close object interactions, partial occlusions, and constrained manipulation spaces. As the assembly structure gradually grows, visible object regions may become increasingly limited, making coarse bounding-box representations insufficient for reliable manipulation. Accurate segmentation masks allow robotic systems to better estimate object contours, contact regions, and occlusion boundaries, thereby improving the robustness of grasp generation, collision checking, and insertion planning.

Recently, general-purpose segmentation frameworks such as the Segment Anything Model (SAM) [21] have demonstrated strong segmentation performance across diverse visual tasks. SAM can generate high-quality pixel-level segmentation masks from prompts such as points, bounding boxes, or coarse masks. In robotic perception pipelines, SAM is commonly integrated with open-vocabulary object detectors to form Grounded-SAM-style perception systems [35]. In such pipelines, language-guided detectors such as GroundingDINO first localize target objects using semantic prompts, and the predicted bounding boxes are subsequently used as prompts for SAM to generate more accurate instance masks.

Compared with standard object detection methods based only on bounding boxes, instance segmentation provides significantly richer geometric information for robotic assembly. The resulting masks can be further used for point cloud extraction, contact analysis, pose estimation refinement, and collision-aware manipulation planning. Therefore, instance segmentation has become an important intermediate representation in modern robotic assembly perception systems.

2.2.3 6D Pose Estimation

To enable robots to grasp and manipulate objects under varying spatial configurations, robotic assembly systems require accurate 6D object pose estimation, including three-dimensional

translation and rotation information. In robotic manipulation and assembly tasks, 6D pose estimation provides essential spatial information for grasp generation, object alignment, placement computation, collision checking, and motion planning. Moreover, accurate object pose estimation is particularly important for future robotic tabletop applications involving object sorting, cluttered manipulation, and irregular object overlap scenarios.

Traditional pose estimation approaches often estimate object poses using geometric reconstruction from depth observations and point cloud analysis [16, 48]. By combining depth images with instance segmentation masks, robotic systems can reconstruct partial object geometry and estimate object orientations from geometric features. However, such geometry-based methods are often sensitive to partial occlusions, sensor noise, sparse observations, and object symmetries, which frequently occur in robotic assembly environments. In addition, many traditional geometric registration and correspondence-based methods require iterative optimization or dense point cloud matching, leading to relatively high computational cost and slow inference speed in complex scenes.

To improve robustness, recent robotic manipulation systems increasingly employ CAD-based 6D pose estimation methods that explicitly incorporate known object geometry during pose inference. Such approaches are particularly suitable for robotic assembly tasks because assembly parts usually have predefined geometric models and well-defined structural shapes. By matching RGB-D observations against known CAD models, the system can estimate more accurate object poses even in partially occluded scenes.

FoundationPose is a representative CAD-based pose estimation framework that estimates object poses from RGB-D observations together with object CAD models [45]. The framework supports both 6D pose estimation and pose tracking, while also allowing model-based and model-free object representations. As illustrated in Figure 2.2a, FoundationPose receives either textured CAD models or a small number of reference RGB images as object representations. Based on RGB-D observations from the current scene, the system estimates the 6D pose of the target object and can subsequently track the pose during manipulation or camera motion.

The relevance of FoundationPose for this thesis is practical rather than architectural: LEGO Duplo parts have known CAD geometry, and the robot requires metric object poses rather than only image-space detections. CAD-based pose estimation therefore provides the transformation needed for grasp planning, placement computation, and collision-aware execution.

Figure 2.2b further illustrates an example of 6D pose estimation results, where the predicted object coordinate axes are aligned with the detected object instance in the scene. By obtaining accurate 6D poses of target parts, robotic systems can subsequently perform grasp planning, placement computation, collision checking, and motion planning for downstream assembly operations.

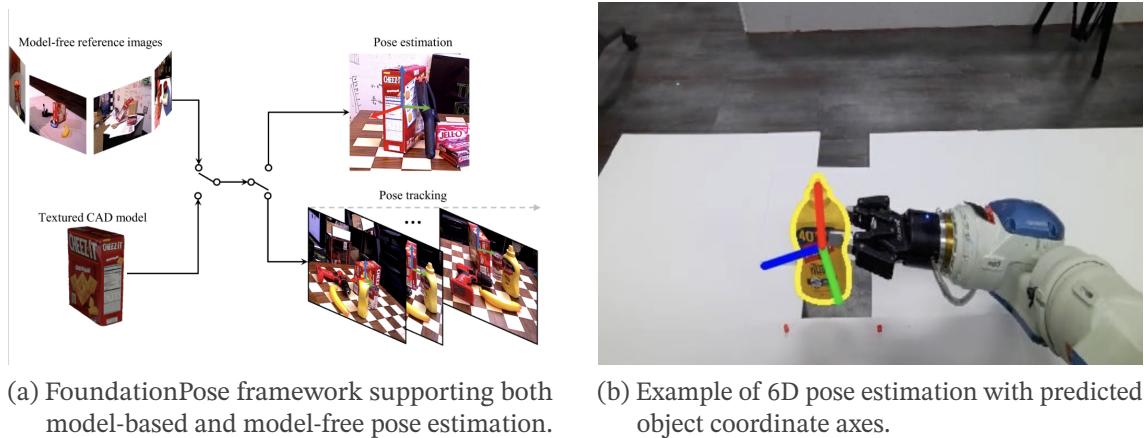


Figure 2.2: Overview of FoundationPose and example 6D pose estimation results. Left: unified model-based and model-free pose estimation framework. Right: example predicted object pose with aligned coordinate axes. Adapted from [45].

2.3 Robotic Execution and State-Machine Control

In robotic assembly systems, symbolic assembly plans must eventually be transformed into executable robot motions and physical interactions. Unlike high-level task planning, execution involves continuous motion generation, collision avoidance, grasp control, insertion alignment, and interaction with partially assembled structures. Therefore, robotic assembly execution is commonly modeled as a sequential manipulation process composed of multiple execution stages.

Finite State Machines (FSMs) are widely used in robotic manipulation and assembly because they provide a structured way to organize sequential robot behaviors and coordinate transitions between different execution stages [6, 9]. In assembly tasks, manipulation actions such as grasping, transport, alignment, insertion, and release often require different motion constraints and control strategies. State-machine-based execution therefore allows robotic systems to decompose complex assembly procedures into smaller and more manageable execution units while maintaining explicit transition logic between different manipulation stages.

A typical assembly operation is usually divided into phases such as approach, grasping, lifting, transport, alignment, insertion, and release. Different stages may require different motion constraints and control strategies. For example, free-space transport is commonly performed using collision-aware trajectory planning, while insertion operations often require constrained Cartesian motions and more precise geometric alignment. As a result, robotic assembly execution is typically organized as a structured sequence of manipulation states rather than a single atomic action.

In a state-machine framework, each manipulation stage is represented as an individual execution state together with explicitly defined transition conditions between states. The robot

switches from one state to another according to the current execution result and workspace condition. For example, after reaching a pre-grasp pose, the system transitions to the grasping state; after successful grasping, the robot enters the transport state; and after reaching the placement location, the execution switches to insertion and release operations.

State-machine execution provides several advantages for robotic assembly. First, it decomposes complex manipulation tasks into smaller and more manageable execution units, making the overall system easier to organize and debug. Second, different control strategies can be assigned to different states. For example, free-space transport motions are commonly executed using collision-aware trajectory planning, while insertion operations often require constrained Cartesian motions [18] and more accurate geometric alignment. Third, the state-machine structure naturally supports execution monitoring and failure recovery.

Failure handling is particularly important in robotic assembly because errors frequently occur during grasping, alignment, or insertion. Grasping may fail due to inaccurate pose estimation or insufficient clearance, while insertion may fail because of geometric misalignment or unexpected contact interactions. In a state-machine framework, the system can locally retry failed actions, switch to alternative grasp configurations, or trigger replanning procedures without restarting the entire task.

2.4 ROS and MoveIt

Modern robotic assembly systems typically require the integration of perception, planning, motion generation, and low-level robot control within a unified software framework. Robot Operating System 2 (ROS 2) has become one of the most widely adopted middleware frameworks in robotics because it provides standardized communication interfaces and modular software composition for distributed robotic systems [29]. In ROS 2, different functional modules such as perception, task planning, motion planning, and execution control can operate as independent nodes while exchanging data through topics, services, and actions. This modular architecture enables robotic assembly systems to flexibly integrate heterogeneous sensing, planning, and control components.

MoveIt is a widely used robotic motion planning framework built on top of ROS [10, 11]. It provides motion planning, kinematics computation, collision checking, and trajectory execution functionalities for robotic manipulation tasks. Given a target robot pose or joint configuration, MoveIt generates collision-free trajectories while considering robot kinematics, joint limits, and workspace constraints. To achieve this, MoveIt maintains a continuously updated planning scene that contains robot states, workspace geometry, and collision objects, allowing the planner to evaluate motion feasibility in dynamically changing environments.

Motion planning in MoveIt is commonly performed using the Open Motion Planning Library (OMPL) [37], which provides sampling-based motion planning algorithms for high-dimensional robotic systems. The resulting trajectories can subsequently be time-parameterized and executed by robot controllers. In robotic assembly tasks, collision-aware motion planning is particularly important because the robot must avoid collisions not only with the workspace and surrounding objects, but also with partially assembled structures that gradually increase the complexity of the environment during execution.

Due to their modularity, extensibility, and strong community support, ROS 2 and MoveIt have become standard software tools in robotic manipulation and assembly research. They provide a unified infrastructure for integrating perception, planning, and execution modules in both simulation and real-world robotic systems.

3 Related Work

This chapter reviews prior work related to the main components of the proposed system: assembly-sequence planning, physics-aware feasibility evaluation, perception for robotic assembly, low-level execution, and integrated robotic assembly pipelines. The emphasis is on methods that connect high-level assembly reasoning with geometric or physical constraints, since this connection is central to reliable LEGO Duplo assembly.

3.1 Assembly-Sequence Planning

Assembly-sequence planning has long been studied as a central problem in robotic assembly because the assembly order directly affects both execution feasibility and computational complexity. Early approaches formulated the problem using symbolic assembly relations, blocking graphs, and geometric precedence constraints to determine feasible assembly paths [14, 30, 36, 46]. In these methods, assembly feasibility is represented through directional or non-directional blocking relationships between parts. A component can only be assembled or removed if a collision-free motion exists along a valid direction or through a feasible path.

Homem de Mello and Sanderson [30, 31] proposed one of the earliest Assembly-by-Disassembly (ABD) formulations, where feasible assembly orders are obtained by reversing valid disassembly sequences. Their work demonstrated that disassembly reasoning can naturally encode geometric precedence constraints while reducing the branching factor compared with forward assembly search. Halperin et al. [14] introduced non-directional blocking graphs to analyze geometric interference relationships between parts and reduce the assembly planning problem into graph reasoning over feasible disassembly motions.

However, these early approaches were typically restricted to relatively simple geometries and idealized assembly conditions because the search complexity increases rapidly as the number of assembly components grows. Most methods primarily focused on symbolic or purely geometric feasibility while assuming simplified contact interactions, fixed insertion directions, or gravity-free settings.

To handle more complex assemblies, later work incorporated geometric collision checking, continuous motion planning, and physics-based reasoning into the planning process [23, 39, 41, 51]. Instead of considering only symbolic ordering constraints, these methods additionally evaluate collision-free assembly paths, insertion directions, object interactions, and robot kinematic constraints during sequence generation.

Sundaram and colleagues [39] integrated geometric motion constraints directly into assembly planning by evaluating feasible translational motions in configuration space. Subsequent approaches further introduced sampling-based motion planning and constraint-based search strategies to generate executable assembly trajectories under narrow-clearance conditions [51]. Rather than treating assembly as a purely symbolic search problem, these methods explicitly reason about continuous spatial feasibility during each assembly transition.

Physics-based assembly planning has also attracted increasing attention in recent years. Tian et al. [41] combined physical simulation with assembly sequence generation to evaluate contact-rich interactions and structural feasibility during intermediate assembly states. Such methods attempt to capture effects including gravity, friction, support stability, and dynamic contact forces that are difficult to model using purely symbolic or geometric reasoning alone.

In real robotic assembly tasks, gravitational stability and structural support relations must be considered throughout the planning process. A sequence that is symbolically valid may still fail during execution if intermediate structures become unstable, unsupported, or physically unreachable. However, accurately evaluating the physical feasibility of complex assemblies remains computationally expensive because each candidate assembly transition may require repeated collision checking, motion planning, contact analysis, and stability verification.

To improve planning efficiency, several approaches employ hierarchical feasibility evaluation strategies, where simpler static constraints are evaluated before more expensive motion-planning or physics-based constraints are applied [20, 23, 41]. This design is also relevant to the proposed system: inexpensive voxel and grasp-clearance checks are applied during search, while full robot motion planning is deferred to the execution stage.

3.1.1 Physics-Based Simulation for Assembly

Beyond purely geometric analysis, physics-based simulation has increasingly been adopted to evaluate the physical feasibility of assembly and disassembly sequences. Unlike symbolic or purely geometric planners, physics-based methods explicitly model contact interactions, gravity, friction, and structural stability during manipulation. This allows robotic systems to reason not only about whether a motion is geometrically feasible, but also whether the resulting intermediate assembly states remain physically stable throughout execution.

Early work incorporated physics engines into disassembly planning primarily for stability analysis. Aleotti and Caselli [2] employed physics-based animation and rigid-body simulation to evaluate the stability of intermediate disassembly states and guide sequence generation. Similarly, Rakshit and Akella [34] analyzed how motion paths and assembly sequences affect the stability of assembled structures under gravitational constraints. However, these studies mainly focused on relatively simple block-stacking scenarios and did not consider large-scale multi-part robotic assembly under realistic manipulation constraints.

Later work explored the integration of physics simulation with search-based planning to improve computational efficiency and physical realism. Kim and Likhachev [20] proposed simulation-aided physical reasoning for assembly planning under uncertainty, where physically plausible state graphs were constructed to accelerate feasibility search. Their framework was evaluated on insertion-oriented tasks such as box-on-table placement and peg-in-hole assembly. Rather than exhaustively validating every candidate action using full simulation, the planner selectively applied physics reasoning only to promising transitions, reducing computational cost.

Physics simulation has also become an important component in reinforcement-learning-based robotic assembly systems. Several studies combined simulation environments with reinforcement learning to learn insertion and manipulation policies for contact-rich tasks [28, 47, 50]. RoboAssembly [50], for example, introduced a multi-robot contact-rich simulation environment for learning furniture assembly policies. Other approaches learned force-guided insertion skills and demonstrated assembly behaviors directly in simulation before transferring them to real robotic platforms. However, many of these methods focus primarily on low-level manipulation primitives such as peg-in-hole insertion, lap-joint assembly, or local alignment correction rather than long-horizon multi-part assembly planning.

More recently, Signed Distance Field (SDF)-based simulation frameworks have been introduced to model contact-rich assembly interactions more accurately. Compared with traditional convex-hull approximations, SDF-based representations provide smoother contact modeling and more stable gradient-based optimization for narrow-clearance insertion tasks. Such methods improve the simulation of frictional contact, interlocking geometry, and continuous collision response during assembly. Nevertheless, despite the progress in physically realistic simulation, many existing approaches still focus on isolated insertion skills or simplified manipulation settings rather than generating complete assembly sequences that are simultaneously collision-free, structurally stable, and executable on real robotic systems.

3.1.2 Learning-Based Assembly Planning

Learning-based assembly planning aims to reduce the combinatorial search cost by learning assembly decisions from data. Instead of enumerating all possible part orders, these methods train models to predict promising assembly or disassembly actions from the current assembly state.

Early methods used self-supervised data collection, where valid and invalid assembly transitions were obtained through repeated trial-and-error interactions. These data-driven strategies can reduce manual rule design, but they were mostly applied to simplified shape-matching or kit-assembly tasks, where geometry and contact conditions are relatively simple [44].

Graph-based methods are especially suitable for assembly planning because an assembly can be represented as a relational graph. Parts are modeled as nodes, while edges encode contact relations, support dependencies, blocking relations, or geometric constraints. A graph neural network can then learn structural dependencies between parts and predict which component should be assembled or disassembled next. Watanabe and Inada [44] used previously collected assembly sequence data to guide sequence search, reducing the need for exhaustive enumeration. Kitaz and Thomas [22] proposed neural dynamic assembly sequence planning, where the model predicts assembly decisions from changing intermediate assembly states rather than from a fixed final structure alone.

Reinforcement learning formulates assembly planning as a sequential decision-making problem. The state usually encodes the current partial assembly, the available parts, and sometimes geometric or contact information. The action corresponds to selecting the next part, pose, or assembly operation. The reward is designed to encourage successful assembly progress and penalize invalid transitions, collisions, or unstable states. Funk et al. [13] combined graph-based reinforcement learning with mixed-integer programming for 3D robot assembly discovery, using graph representations to reason about part relations while optimization constraints help enforce feasible assembly decisions.

Although learning-based methods can improve inference efficiency and capture structural patterns from data, they often require large training datasets or extensive simulation. They may also have difficulty strictly enforcing hard geometric constraints, such as collision-free insertion paths, gripper clearance, and intermediate structural stability. For this reason, learning-based planners are often combined with explicit feasibility checks when reliable robotic execution is required.

3.2 Perception for Robotic Assembly

Perception is a fundamental component of robotic assembly because robots must reliably identify, localize, and track assembly parts under cluttered and partially occluded conditions. Compared with traditional pick-and-place tasks, robotic assembly often requires significantly higher geometric precision since small localization errors can directly lead to grasp failures, insertion collisions, or unstable assemblies.

3.2.1 Classical Vision-Based Assembly Perception

Early robotic assembly systems mainly relied on handcrafted vision pipelines based on color segmentation, edge extraction, template matching, or geometric feature detection. Such approaches were widely used in structured industrial environments where object appearance, lighting, and camera viewpoints were tightly controlled. Classical feature-based methods further employed SIFT, SURF, or contour matching together with RGB-D sensing and point-cloud

registration to estimate object poses [16]. Although these methods can provide accurate geometric alignment in constrained settings, they are highly sensitive to occlusions, illumination changes, reflective surfaces, and sensor noise.

3.2.2 Deep Learning-Based Perception

With the development of deep learning, convolutional neural network-based detectors such as Faster R-CNN, YOLO, and Mask R-CNN became widely adopted for robotic manipulation and assembly perception. These methods significantly improved robustness under cluttered tabletop environments by learning semantic object representations directly from large-scale datasets. In robotic assembly, object detection is commonly combined with instance segmentation to obtain more accurate object boundaries for grasp synthesis and collision reasoning. For example, Mask R-CNN-based systems have been used to segment assembly parts before performing pose estimation and grasp planning in industrial manipulation tasks.

More recent work has adopted real-time segmentation-based detectors such as YOLOv8-seg for robotic assembly environments. Barghi et al. [3] employed a YOLOv8m-seg model for LEGO brick localization and segmentation in collision-aware LEGO reconstruction tasks. Their system fine-tuned the detector on a custom dataset containing LEGO bricks with multiple orientations and viewpoints to improve robustness under cluttered assembly scenes. However, they observed that coarse bounding-box localization alone is insufficient for reliable grasp generation because grasp accuracy strongly depends on stud alignment and brick orientation estimation.

To improve localization precision, several works combine deep visual detection with geometric reasoning. In the system of Barghi et al. [3], OpenCV-based circle detection was applied to detect LEGO studs after instance segmentation. The detected stud coordinates were subsequently used to estimate brick orientation and grasp positions. Similar hybrid approaches combining deep segmentation with geometric keypoint extraction have also been explored in peg-in-hole assembly and precision insertion tasks, where accurate contact alignment is required.

3.2.3 6D Pose Estimation

Beyond 2D localization, robotic assembly systems usually require accurate 6D object pose estimation. Traditional pose estimation methods commonly rely on point-cloud registration techniques such as ICP or feature-correspondence matching between RGB-D observations and CAD models [48]. However, these methods often degrade under partial occlusions or sparse depth observations. To address these limitations, learning-based pose estimation frameworks such as PoseCNN [48], DenseFusion [43], and FoundationPose [45] have been proposed. DenseFusion combines RGB features and depth point-cloud features through iterative fusion to improve robustness in cluttered scenes, while FoundationPose directly leverages CAD

geometry together with RGB-D observations for robust pose estimation and tracking during manipulation.

Recent advances in vision-language models have further improved perception flexibility in robotic assembly. Open-vocabulary detectors such as GroundingDINO [27] can localize objects directly from natural-language prompts rather than fixed object categories. Combined with SAM [21], these systems enable Grounded-SAM-style perception pipelines that jointly perform semantic grounding and pixel-level segmentation. Such methods are particularly useful for robotic assembly because target parts are often specified through semantic task descriptions instead of predefined detector labels.

However, robotic assembly perception remains challenging because assembly tasks frequently involve severe occlusions, contact-rich interactions, reflective materials, dense object arrangements, and strict geometric tolerances. Even small segmentation or pose-estimation errors may propagate into downstream planning and execution failures. Consequently, modern robotic assembly systems increasingly combine deep visual perception, geometric reasoning, and physical feasibility constraints to improve robustness in long-horizon assembly tasks.

3.3 Low-Level Execution and Task Control

Low-level execution converts assembly plans into physically executable robot motions under geometric, kinematic, and contact constraints. In robotic assembly tasks, execution usually consists of sequential phases such as approach, grasp, lift, transport, alignment, insertion, press-in, release, and retreat. Different phases require different control strategies. Free-space transport mainly depends on collision-free trajectory generation, while insertion and alignment require accurate Cartesian motion, contact handling, and small pose-error correction.

3.3.1 Classical Robotic Assembly Execution

One common execution strategy is to use structured state-machine control [6, 9]. In these systems, the assembly task is decomposed into explicit execution states such as PREGRASP, GRASP, LIFT, PREPLACE, INSERT, PRESS, and RETREAT. Each state defines its own controller, motion constraints, transition conditions, and recovery rules. For example, transport phases are commonly executed using joint-space trajectory planning, while insertion phases use constrained Cartesian motions along predefined insertion directions. State-machine execution also supports local failure recovery, such as retrying grasps, switching to alternative grasp candidates, or replanning failed insertion actions.

Collision-aware motion planning is another important component of low-level assembly execution. Sampling-based motion planners such as OMPL and MoveIt are widely used to generate robot trajectories under kinematic limits, joint constraints, and planning-scene colli-

sion constraints [10, 11, 37]. During assembly, the planning scene must continuously update the workspace geometry, loose parts, and partially assembled structures. As the assembly grows, feasible motion space becomes smaller and narrow-clearance manipulation becomes increasingly difficult.

3.3.2 Visual Servoing and Contact Control

Precise insertion additionally requires closed-loop feedback control. Visual servoing methods are commonly divided into image-based visual servoing (IBVS) and position-based visual servoing (PBVS) [18]. IBVS minimizes errors directly in image feature space, while PBVS estimates the relative target pose and minimizes Cartesian pose errors. Force or impedance control is also often used during insertion to regulate contact forces and avoid excessive contact or jamming.

Besides classical motion-planning pipelines, recent work increasingly explores learning-based low-level execution. Reinforcement learning methods learn insertion, alignment, and contact-rich manipulation policies directly from interaction data. In these approaches, policies map observations and robot states to continuous actions, while reward functions are designed around insertion success, contact stability, pose error, or force minimization. Such methods can learn compliant behaviors that are difficult to model analytically, but they usually require large amounts of simulation data and often generalize poorly under unseen object geometries or friction conditions.

3.3.3 Vision-Language-Action Models

More recently, Vision-Language-Action (VLA) models have been proposed for end-to-end robotic manipulation. Unlike classical robotic systems that explicitly separate perception, planning, and control, VLA systems directly predict robot actions from multimodal observations including RGB images, language instructions, and robot states. Most VLA architectures are transformer-based and learn visuomotor policies from large-scale robot demonstration datasets.

RT-1 [7] introduced a transformer-based robot policy trained on large-scale real-world robot data by tokenizing observations and actions into a sequence prediction problem. RT-2 [8] further incorporated vision-language representations from internet-scale pretrained models, enabling semantic reasoning from natural-language instructions. More recent systems such as OpenVLA [19] and π -style visuomotor policies [5] extend this idea using autoregressive or diffusion-based action generation. These models directly generate continuous robot actions such as end-effector motions or gripper commands without explicitly computing symbolic assembly sequences, grasp poses, or insertion trajectories.

A key characteristic of π -style policies is that manipulation strategies are learned implicitly from demonstration trajectories rather than manually engineered through symbolic planning

rules. The model attempts to infer grasping, alignment, and insertion behaviors directly from multimodal training data. Diffusion-based action decoders are particularly useful because they can generate smooth continuous trajectories and represent multimodal action distributions during contact-rich manipulation.

Although VLA systems demonstrate strong generalization ability for open-world manipulation tasks, robotic assembly remains challenging for purely end-to-end visuomotor execution. Assembly tasks involve long-horizon sequential dependencies, narrow-clearance insertion, contact-sensitive interactions, and strict geometric tolerances. Small pose errors during insertion can accumulate across multiple steps and lead to failure. In addition, many VLA systems do not explicitly enforce collision checking, insertion feasibility, or structural stability during inference, which may produce physically invalid actions even when the generated behavior appears semantically reasonable.

For this reason, recent robotic assembly systems increasingly combine learning-based execution with explicit geometric planning and physical feasibility verification. In such hybrid systems, learned policies provide semantic priors or coarse manipulation guidance, while collision-aware motion planning, grasp feasibility analysis, and state-machine execution ensure reliable physical assembly behavior.

3.4 Integrated Robotic Assembly Pipelines

Recent robotic assembly research has increasingly focused on hybrid frameworks that integrate high-level planning with low-level feasibility verification. Instead of relying only on a symbolic task planner or an end-to-end policy, these systems typically combine discrete task reasoning, geometric motion planning, physical simulation, and execution monitoring within a unified pipeline. This design is especially relevant for assembly tasks, where a high-level plan may be logically valid but still fail during execution because of collisions, unreachable grasps, unstable intermediate structures, or insertion errors.

3.4.1 LLM-Based Task Planning

Some recent approaches use Large Language Models (LLMs) to generate high-level task plans from natural language instructions or scene descriptions [1, 17]. In such systems, the LLM proposes an ordered sequence of actions, while feasibility is checked later through affordance models, motion planning, collision checking, or physics simulation. This approach improves task flexibility and semantic reasoning, but the generated plans still require explicit grounding in robot kinematics, object geometry, and physical constraints.

3.4.2 Vision-Language Reasoning for Assembly

Vision-Language Models (VLMs) extend this idea by incorporating visual feedback into the planning process [8, 19]. These systems can use images or RGB-D observations to identify objects, reason about spatial relations, and guide plan refinement. In assembly tasks, VLMs can help select target parts, detect execution failures, or provide semantic feedback for replanning. However, VLMs usually operate at a semantic or object-relation level and often do not directly guarantee collision-free motion, grasp feasibility, or contact-stable insertion.

3.4.3 Symbolic-Geometric Hybrid Planning

Another line of work combines symbolic or graph-based planning with motion planning and physics simulation [3, 4, 40]. In these hybrid symbolic-continuous architectures, a planner first generates a discrete state graph or action sequence. Each symbolic action is then grounded into continuous robot motions through motion planning and validated using geometric collision checking or physical simulation. If grounding fails, the system can trigger backtracking, symbolic replanning, or local motion replanning. This structure allows the system to combine the clarity of symbolic reasoning with the physical feasibility checks required for real robotic execution.

State-machine execution is also commonly integrated into such frameworks to manage low-level task progress. After a task plan has been generated and grounded, execution is divided into states such as approach, grasp, transport, align, insert, release, and retreat. Each state can use its own controller, success condition, and failure handling logic. This makes it easier to monitor execution and recover from local failures without restarting the entire assembly task.

Overall, existing hybrid frameworks show that reliable robotic assembly requires coordinated integration of symbolic planning, geometric reasoning, grasp feasibility analysis, motion planning, physical validation, and low-level execution control. Different systems emphasize different modules, but robust assembly performance generally depends on connecting high-level task reasoning with continuous feasibility verification and execution feedback.

4 Methods

This chapter describes how the proposed system converts a target LEGO Duplo structure into executable robotic assembly actions. It first introduces the workspace, task representation, and system-level data flow. It then presents the physical feasibility planner, the open-vocabulary perception pipeline, and the collision-aware execution controller used to realize the planned assembly sequence.

4.1 System Overview and Setup

Our approach constructs a structured robotic assembly pipeline that transforms a high-level product specification into executable assembly actions. We consider a vision-guided tabletop assembly setting with a 6-DoF manipulator, a parallel-jaw gripper, and an RGB-D camera. In our experiments, we use a Franka Emika Panda robot arm and evaluate the pipeline in MuJoCo simulation integrated with ROS 2 and MoveIt 2 for motion planning and execution.

The robotic assembly workspace used in this work is shown in Figure 1.2. The workspace is divided into two functional regions: a picking area containing initially separated LEGO Duplo bricks and an assembly area used for target structure construction. Both regions are mounted on LEGO-compatible baseplates to provide consistent geometric alignment during manipulation and insertion. An RGB-D camera observes both areas and provides color and depth observations for perception and pose estimation. The Franka Emika Panda robot arm is positioned between the two regions, allowing the manipulator to transport bricks from the picking area to the assembly area while maintaining collision-free motion within the reachable workspace.

The assembly goal is specified as a structured YAML product description. Each brick entry contains the brick identifier, brick type, color, spatial position, and orientation within the final target structure. This representation defines the desired final geometry before planning starts and provides a compact intermediate format shared between planning, perception, and execution modules.

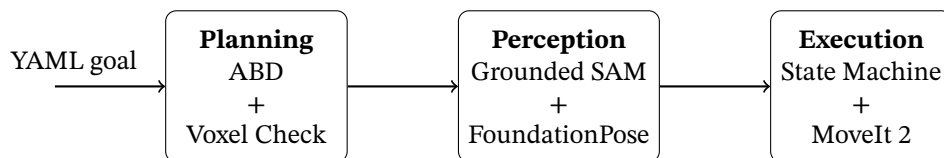


Figure 4.1: System data flow from a structured YAML assembly goal to sequence planning, open-vocabulary perception, and collision-aware robot execution.

Figure 4.1 illustrates the overall three-stage pipeline consisting of planning, perception, and execution modules. The system starts from a structured YAML assembly goal that defines the target structure and part relationships. Based on this specification, an Assembly-by-Disassembly (ABD) planning procedure is applied to reason about feasible disassembly sequences of the target structure. By reversing the resulting disassembly order and incorporating grasp reachability together with press-stability checks, the system derives a physically executable assembly sequence and corresponding candidate grasp configurations for each part. To support efficient geometric reasoning and collision checking, the assembly state is represented using a voxel-aligned occupancy structure consistent with the LEGO stud grid.

Given the planned assembly sequence, the perception module grounds task-level part specifications in the physical scene. GroundingDINO and the Segment Anything Model (SAM) are used to identify target parts and estimate instance-level object masks from RGB-D observations. The resulting instance masks, together with RGB-D data and corresponding CAD models, are subsequently provided to FoundationPose for 6D pose estimation in the camera coordinate frame. Using the calibrated rigid transform between the camera and the robot base, the estimated object poses are transformed into the robot coordinate frame to generate executable grasp targets consistent with the planned grasp configurations.

Finally, the execution module realizes the planned assembly actions through collision-aware motion planning and structured state-machine execution. Based on the perceived object poses, MoveIt 2 generates collision-free trajectories for approach, grasp, transport, placement, insertion, and retreat motions under kinematic and workspace constraints. Contact-sensitive insertion and press-in stages are executed using constrained Cartesian motions to maintain accurate stud alignment during assembly. The state-machine framework additionally supports execution monitoring and local failure recovery, including grasp retries and replanning under failed insertion or collision conditions. Closed-loop visual servoing can further be integrated as a refinement mechanism when higher insertion precision is required.

Listing 1 shows an example `final_product.yaml` specification describing a stacked LEGO Duplo structure. Based on this structured representation, the planning module reconstructs the voxel-aligned assembly state, infers structural dependencies between bricks, and generates a physically executable assembly sequence through ABD reasoning. The positions are represented in a grid-aligned metric coordinate system consistent with the LEGO stud layout.

4.2 Assembly-Sequence Planning with Physical Feasibility

Assembly-by-Disassembly Planning The planning module receives a target product description from the structured YAML specification and converts it into an executable assembly sequence. The target structure is represented as a discrete voxel state aligned with the LEGO stud grid. Each voxel cell stores both occupancy and brick identity information, so the planner

```
blocks:
- name: "2x2_brick_1"
  type: "brick_2x2"
  color: "yellow"
  pos: [0.0, 0.148, 0.0]
  rotation: [0, 0, 90]

- name: "2x2_brick_2"
  type: "brick_2x2"
  color: "green"
  pos: [0.032, 0.148, 0.0]
  rotation: [0, 0, 0]

- name: "4x2_brick_3"
  type: "brick_4x2"
  color: "yellow"
  pos: [0.016, 0.116, 0.0]
  rotation: [0, 0, 0]

- name: "2x2_brick_4"
  type: "brick_2x2"
  color: "blue"
  pos: [-0.032, 0.116, 0.0]
  rotation: [0, 0, 0]

- name: "2x2_brick_5"
  type: "brick_2x2"
  color: "red"
  pos: [0.0, 0.116, 0.0191]
  rotation: [0, 0, 0]

- name: "2x2_brick_7"
  type: "brick_2x2"
  color: "blue"
  pos: [0.032, 0.148, 0.0191]
  rotation: [0, 0, 0]

- name: "2x2_brick_9"
  type: "brick_2x2"
  color: "blue"
  pos: [0.0, 0.148, 0.0191]
  rotation: [0, 0, 90]
```

Listing 1: Example YAML task specification for a LEGO Duplo assembly. Each block entry defines the part identity, type, color, target position, and target orientation used by the planner.

Algorithm 1 ABD reasoning with grasp reachability and press-stability checks

```

1  function DISASSEMBLE( $S$ )
2    if  $S = \emptyset$  then
3      return []
4     $C \leftarrow \text{TOPDOWNCANDIDATES}(S)$ 
5    for all  $p \in C$  do
6      if REACHABLE( $p, \mathcal{G}(p), w, S$ ) and PRESSSTABLE( $S \setminus \{p\}$ ) then
7        if ISSUBASSEMBLY( $p, S$ ) then
8           $A \leftarrow \text{BUILDSUBASSEMBLY}(p, S)$ 
9           $\pi_{\text{sub}} \leftarrow \text{DISASSEMBLE}(A)$ 
10         if  $\pi_{\text{sub}} = \text{FAIL}$  then
11           continue
12          $\pi_{\text{rest}} \leftarrow \text{DISASSEMBLE}(S \setminus A)$ 
13         if  $\pi_{\text{rest}} = \text{FAIL}$  then
14           continue
15         return  $\pi_{\text{sub}} + \pi_{\text{rest}}$ 
16       else
17          $\pi_{\text{rest}} \leftarrow \text{DISASSEMBLE}(S \setminus \{p\})$ 
18         if  $\pi_{\text{rest}} = \text{FAIL}$  then
19           continue
20         return  $[p] + \pi_{\text{rest}}$ 
21   return FAIL

```

can reason about which space is occupied, which brick owns each occupied cell, and how different bricks are connected inside the target structure. This representation is particularly suitable for LEGO Duplo assembly because the regular stud-grid geometry makes spatial relations, support relations, and local collision checks easier to evaluate than in an unstructured mesh representation.

We formulate assembly-sequence generation as a recursive Assembly-by-Disassembly (ABD) search. Instead of constructing the product forward from an empty assembly area, the planner starts from the completed target structure and reasons about how it can be taken apart. The resulting disassembly sequence is then reversed to obtain the assembly sequence used during execution. This reduces the branching factor compared with forward assembly search, since the planner only considers components that are currently removable from the completed or partially disassembled structure.

Algorithm 1 takes the current structure state S as input and returns a disassembly sequence π , represented as an ordered list of bricks. The final assembly sequence is obtained by reversing π . At each recursive search state S , the planner first generates a set of top-down removable candidates C . This candidate generation follows the physical structure of LEGO assembly: parts that are higher in the structure or not blocked by other bricks are considered before lower supporting components. For each candidate brick $p \in C$, the planner evaluates whether

removing p is geometrically and physically feasible. A candidate is rejected if it is blocked by neighboring bricks, if removing it would break a required connected subassembly, if no feasible grasp configuration is available, or if the remaining structure would fail the press-stability check used for the reversed assembly action.

The first feasibility condition is grasp reachability. For each candidate brick, a finite set of predefined grasp candidates is generated according to the brick geometry and the parallel-jaw gripper configuration. The planner checks whether at least one grasp candidate provides enough free space for the gripper and admits a collision-free approach motion in the current voxel state. In this way, the planner does not only ask whether a brick is symbolically removable, but also whether the robot can physically reach and grasp it during execution.

The second feasibility condition is press-stability. Although the search is performed as disassembly, the final execution is the reversed assembly process. Therefore, when the planner considers removing a brick p , it also checks whether the remaining partial structure $S \setminus \{p\}$ would be stable when p is later inserted by a downward press-in motion. This is important for LEGO Duplo assembly because stud engagement typically requires a vertical pressing force. If the corresponding press operation would topple or destabilize the partially assembled structure, the candidate is rejected even if it is geometrically removable.

If both grasp reachability and press-stability are satisfied, the candidate p is accepted as the next disassembly action. If p belongs to a connected subassembly, the planner constructs the connected component A using `BUILDSUBASSEMBLY` and recursively processes that subassembly as a group. The sequence π_{sub} denotes the local disassembly order inside this subassembly, while π_{rest} denotes the disassembly order for the remaining structure. If either recursive call returns `FAIL`, the algorithm backtracks by continuing to the next candidate in C . The search therefore proceeds in a depth-first manner and returns the first valid disassembly sequence π found under the reachability and stability checks.

Figure 4.2 illustrates this ABD reasoning process on an example target structure. The numbered bricks indicate the inferred disassembly order, and the black markers show candidate grasp-point pairs evaluated during planning. Algorithm 1 summarizes the recursive procedure. By filtering infeasible candidates during disassembly, the planner generates an assembly sequence that already accounts for grasp reachability, local collision constraints, and press-stability requirements before execution.

Lines 2–4 define the recursion base case: an empty structure has an empty disassembly sequence. Line 5 generates the candidate set C . Lines 7–31 test candidates against grasp reachability and press-stability, recursively solve subassemblies when necessary, and backtrack when a recursive branch fails. The use of list concatenation in lines 20 and 29 records the order in which parts are removed.

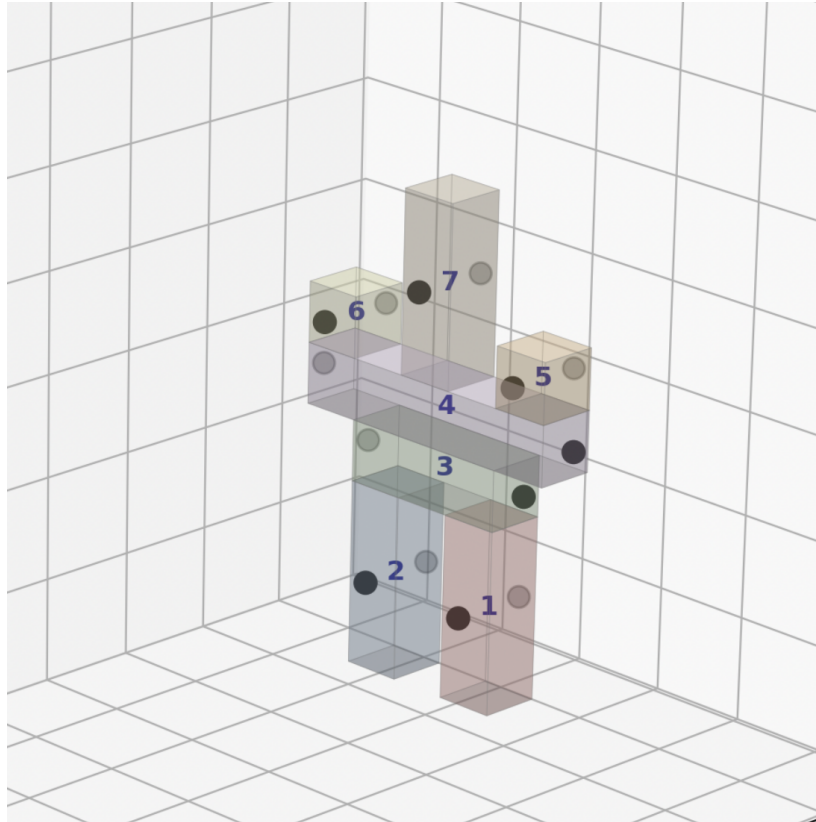


Figure 4.2: Example of ABD reasoning with inferred disassembly order and candidate grasp-point pairs. The numbered blocks indicate the inferred removal sequence, while the black markers represent candidate grasp-point pairs evaluated during planning.

Grasp Reachability Analysis For each candidate block p , the planner evaluates whether the grasp action can be executed under the current workspace configuration. Instead of testing only a single grasp pose, the system defines a finite set of candidate grasp-point pairs $\mathcal{G}(p) = \{g_1, g_2, \dots, g_n\}$, where each g_i corresponds to a pair of opposing contact points compatible with the parallel-jaw gripper. These grasp candidates are generated according to the brick geometry and represent different side-grasp directions around the target brick.

To efficiently approximate manipulation feasibility during planning, each grasp candidate is associated with a gripper occupancy region representing the workspace volume occupied by the gripper fingers and gripper body during grasp execution. Rather than performing expensive mesh-level collision checking, the occupancy region is discretized directly in the voxel-state representation.

In our implementation, the gripper occupancy region is modeled as a fixed 2×2 voxel area aligned with the LEGO stud grid, corresponding to four occupied grid cells surrounding the target grasp location. Candidate grasps are generated mainly from opposing long-side contact pairs because these grasps match the parallel-jaw gripper geometry and leave the top surface

available for vertical press-in operations. Short-side or top grasps are excluded in the current implementation because they either provide less stable contact for elongated Duplo bricks or interfere with the planned insertion direction. This representation provides a conservative approximation of the minimum free space required for collision-free grasp execution.

During evaluation, the planner checks whether the occupancy region overlaps with neighboring bricks, surrounding objects, or partially assembled structures under the current voxel occupancy state. If any overlap exists, the corresponding grasp candidate is rejected because the gripper would collide with the environment during grasp execution. Consequently, a grasp candidate is considered feasible only when sufficient free clearance exists around the target brick.

Figure 4.3 illustrates two examples of grasp feasibility evaluation under voxel-based occupancy constraints. The dashed regions visualize the approximated gripper occupancy area considered during planning. In the left example, the occupancy region remains collision-free, allowing the grasp candidate to be accepted. In the right example, the occupancy region intersects with neighboring bricks, making the grasp physically infeasible under the current scene configuration.

During recursive ABD planning, grasp reachability acts as an execution-aware feasibility constraint. A candidate block p is considered removable only if at least one grasp candidate $g_i \in \mathcal{G}(p)$ satisfies the collision-free occupancy condition. Otherwise, the candidate is rejected and excluded from the recursive disassembly search.

Press-Stability Check In addition to grasp feasibility, the planner evaluates whether the partial assembly remains stable during the reversed insertion process. This check is motivated by the physical characteristics of LEGO-style assembly, where bricks are typically connected through a downward press-in motion to engage studs and complete the attachment. Although a candidate part may be geometrically insertable, the insertion force itself can destabilize the partially assembled structure and cause toppling during execution.

Instead of performing computationally expensive multi-step physics simulation during recursive ABD search, we employ a lightweight quasi-static geometric approximation for stability evaluation. This approximation is particularly suitable for LEGO Duplo assembly because the insertion direction is predominantly vertical and the stud-grid geometry provides well-defined support relationships.

Let S denote the current partial assembly structure. We define the support region $\mathcal{P}$ as the planar support area generated by all ground-contacting bricks in the current structure. In our implementation, $\mathcal{P}$ is approximated from the projected footprints of all bottom-layer bricks on the supporting plane, followed by a convex-hull construction.

For a candidate insertion operation, the projected press location is denoted as p_{proj} , which represents the planar location where the downward insertion force is applied. A candidate

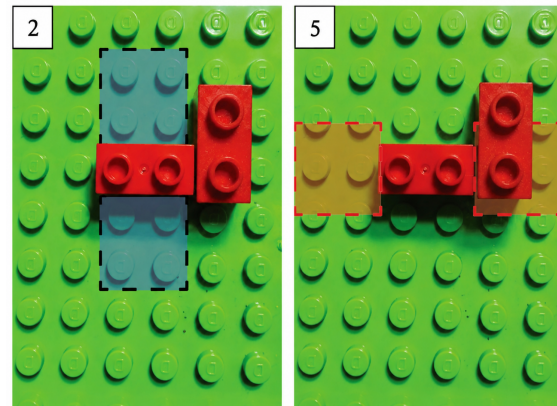


Figure 4.3: Examples of grasp reachability evaluation under voxel-based gripper occupancy constraints. The dashed regions visualize the approximated gripper occupancy area represented as a 2×2 voxel region aligned with the LEGO stud grid. A grasp candidate is considered feasible only if the occupancy region remains collision-free under the current scene configuration. Left: feasible grasp candidate with sufficient free clearance. Right: infeasible grasp candidate where the occupancy region intersects with neighboring bricks. The numbers indicate the candidate evaluation order during ABD search.

insertion is considered press-stable only if $p_{\text{proj}} \in \mathcal{P}$. This condition ensures that the downward press-in force acts within the support boundary, reducing the risk of generating an overturning moment during insertion. If the projected press location falls outside the support region, the corresponding candidate action is rejected because the insertion may destabilize the partially assembled structure during the press-in operation.

Although simplified, this geometric stability model provides an efficient execution-aware feasibility filter during recursive ABD planning. By incorporating press stability directly into candidate evaluation, the planner avoids assembly sequences that are geometrically valid but physically unstable during the final press-in operation.

Subassembly Reasoning During recursive ABD search, the planner evaluates candidate bricks sequentially according to the current removable candidate set. For each candidate brick p , the planner first checks whether it satisfies both feasibility conditions introduced previously: grasp reachability and press-stability. If both conditions are satisfied, the brick can be selected as the current disassembly action. If either condition fails, the planner skips the current candidate and continues evaluating the remaining candidates in the normal search order.

During later search steps, a supporting brick may satisfy both grasp reachability and press-stability constraints while simultaneously forming a connected structure together with previ-

ously skipped upper-layer bricks. In such cases, the planner treats the connected structure as a temporary subassembly instead of reasoning about each brick independently.

This mechanism is particularly important for hierarchical LEGO structures in which upper-layer bricks depend on lower supporting components. An upper brick may initially be infeasible because of insufficient grasp clearance or press-stability violations. However, when the corresponding supporting brick becomes removable, the connected structure can subsequently be processed together as a valid subassembly.

For a detected subassembly $A \subseteq S$, the planner recursively applies the ABD procedure within the connected structure and generates a local disassembly order for the subassembly itself. Since the final assembly sequence is obtained by reversing the disassembly order, removing the supporting brick before the dependent upper brick naturally produces an assembly order in which the supporting structure is assembled first, followed by the upper dependent brick. This preserves the required physical support relationship during execution.

By incorporating subassembly reasoning into recursive ABD search, the planner can handle local structural dependencies without modifying the normal candidate traversal order. This reduces unnecessary search failures and improves the physical executability of the final assembly sequence.

4.3 Perception: Open-Vocabulary Visual Pose Estimation

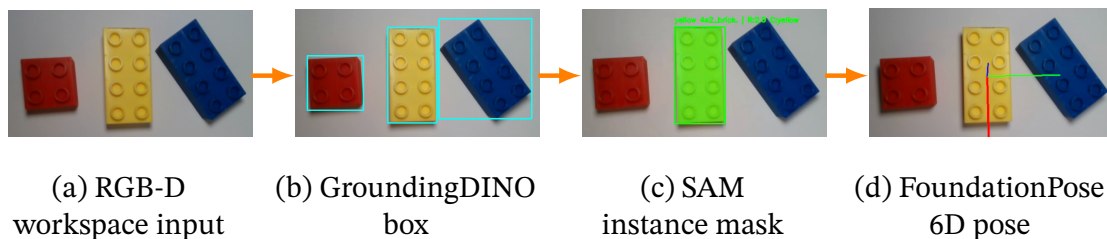


Figure 4.4: Perception pipeline for active part localisation in a LEGO Duplo workspace. From left to right: RGB-D workspace input, GroundingDINO box, SAM instance mask, and FoundationPose 6D pose.

To support robotic grasping, placement, and insertion during assembly execution, the system must continuously localize target LEGO Duplo parts and estimate their spatial poses from RGB-D observations. In this work, we implement a task-oriented open-vocabulary perception pipeline that combines GroundingDINO, SAM, and FoundationPose for semantic object localization, geometric refinement, and CAD-based 6D pose estimation.

The overall perception workflow is illustrated in Figure 4.4. The system receives aligned RGB and depth observations from an RGB-D camera. The depth stream is aligned with the RGB image so that each image region corresponds to metric depth measurements. Camera

intrinsic matrix K , while the calibrated camera-to-base transformation is loaded from a YAML configuration file for downstream robot execution.

During execution, the current assembly task is first loaded from a structured YAML task specification containing the target brick identity and placement configuration. Unlike conventional closed-set detectors trained on fixed object categories, our system adopts open-vocabulary detection to support flexible task-level object specification. GroundingDINO is therefore queried using semantic prompts such as “lego block” to localize candidate LEGO parts inside the workspace.

The detector outputs candidate bounding boxes together with confidence scores. Since different LEGO bricks have characteristic geometric proportions, the system further performs a lightweight geometric verification step before pose estimation. Each candidate region is refined using SAM-based instance segmentation, and the resulting mask is used to estimate the object aspect ratio through oriented bounding-box fitting. Candidates with inconsistent geometric proportions are rejected directly, significantly reducing false detections before downstream pose estimation.

After filtering, the remaining candidates are ranked according to both detection confidence and geometric consistency, and the highest-scoring candidate is selected for 6D pose estimation.

The system subsequently performs CAD-based pose estimation using FoundationPose. Since LEGO Duplo geometries are predefined, the corresponding STL mesh is dynamically loaded according to the current task. Before registration, the mesh is automatically normalized and recentered. Large meshes are rescaled from millimetres to metres when necessary, and the mesh centroid is shifted to the origin so that the object coordinate frame is aligned with the geometric center of the brick.

FoundationPose receives the RGB image, depth image, camera intrinsics, segmentation mask, and CAD mesh as input. Surface points are sampled directly from the CAD geometry and used for iterative RGB-D pose registration. During optimization, the framework aligns rendered CAD observations with the segmented RGB-D observation to estimate the full object pose $T_{\text{cam}}^{\text{obj}} \in \mathbb{R}^{4 \times 4}$, which represents the rigid transformation from the object frame to the camera frame.

Compared with classical ICP-based registration methods, FoundationPose provides significantly improved robustness under partial occlusions, sparse depth observations, varying viewpoints, and cluttered tabletop environments. An additional advantage of FoundationPose is that it supports both pose estimation and online pose tracking within a unified framework. This is particularly useful for robotic assembly because the robot continuously manipulates and reorients objects during execution.

To improve estimation stability, the system continuously tracks the selected object for several seconds and accumulates multiple pose predictions instead of relying on a single-frame estimate. During tracking, the estimated object coordinate axes are projected back into the RGB image for online visualization and debugging. The final stable pose estimate is subsequently transformed into the robot base frame using the calibrated camera extrinsic matrix: $T_{\text{base}}^{\text{obj}} = T_{\text{base}}^{\text{cam}} T_{\text{cam}}^{\text{obj}}$. The transformed object pose is then used for downstream robotic manipulation. The system extracts the object yaw angle from the estimated rotation matrix and normalizes it according to LEGO symmetry constraints. Based on the current assembly strategy, the perception module computes grasp orientations, placement orientations, and insertion alignment targets for the robot controller.

Finally, the resulting object pose, grasp orientation, and placement target are exported through a shared YAML interface to the robot execution node. In this way, the perception module serves as the geometric bridge between RGB-D scene understanding and collision-aware robotic assembly execution.

4.4 Execution: Collision-Aware State Machine

4.4.1 Overall Execution Pipeline

The execution module transforms the planned assembly targets and perceived object poses into physically executable robot motions. In our implementation, execution is realized as a ROS 2 and MoveIt 2 based state-machine controller. The controller receives one active assembly task at a time from the perception module through a shared YAML interface and executes a fixed pick-and-place routine for the corresponding LEGO Duplo brick.

Each active task contains the brick name, the estimated brick pose, the robot pick orientation, and the target placement pose. The brick position is obtained from the perception module, while the pick and place orientations are computed according to the detected brick yaw, the selected grasp strategy, and the target assembly blueprint. This separation allows the system to distinguish between the physical pose of the brick in the scene and the end-effector pose required for grasping.

The execution node continuously monitors the task file. Once a new task is detected, the node parses the YAML file, selects the corresponding STL mesh according to the brick type, and inserts the brick model into the MoveIt planning scene. In our implementation, both 2×2 and 2×4 LEGO Duplo meshes are supported. The mesh geometry is loaded from an STL file and scaled from millimetres to metres before being used as a collision object in the planning scene.

Before execution, the workspace table is also added as a collision object. The planning scene therefore contains the robot model, the table, the target brick, and the relevant workspace

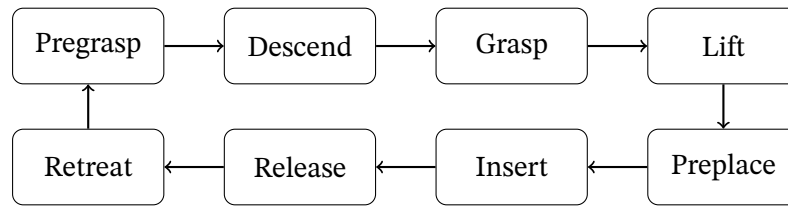


Figure 4.5: Execution state machine for one pick-and-place assembly step. The controller moves from free-space transport to slow insertion, release, and retreat before the next planned part is executed.

geometry. This allows MoveIt 2 to generate collision-aware motions while considering the robot arm, gripper, object mesh, and surrounding environment.

The manipulation routine is organized into the following execution states:

- PREGRASP
- DESCENDTOGRASP
- GRASP
- LIFT
- PREPLACE
- DESCENDTOPLACE
- RELEASE
- RETREAT

The overall execution flow is illustrated in Figure 4.5. First, the robot moves to a pre-grasp pose above the detected brick. This pose is computed by adding a vertical hover offset to the grasp pose, taking into account the gripper height and a predefined safety clearance. The motion to the pre-grasp pose is executed as a point-to-point motion using MoveIt 2. For these large free-space motions, the system uses relatively high velocity scaling to reduce execution time while still relying on planning-scene collision checking.

4.4.2 Grasping and Placement Execution

After reaching the pre-grasp pose, the robot opens the parallel-jaw gripper and performs a slow vertical Cartesian descent toward the brick. The descent is generated using MoveIt Cartesian path planning through `computeCartesianPath`. The trajectory is subsequently time-parameterized using Time-Optimal Trajectory Generation (TOTG), with a low velocity and acceleration scaling factor. This ensures that the final approach remains controlled and reduces the risk of pushing or toppling the target brick before grasping.

Once the gripper reaches the grasp pose, the system closes the gripper through the ROS 2 `GripperCommand` action interface. The brick is then attached to the robot hand in the MoveIt planning scene using the gripper links as touch links. This attachment step is important because subsequent motion planning must treat the grasped brick as part of the robot. After

grasping, the robot lifts the brick vertically using a Cartesian motion with a higher velocity scaling factor than the descent phase.

The robot then moves to a pre-place pose above the target assembly location. Similar to the pre-grasp motion, this transport phase is executed as a MoveIt point-to-point motion in joint space. After reaching the target area, the robot performs a slow Cartesian descent toward the placement pose. This phase uses an even lower velocity scaling factor than the pick descent because accurate stud alignment is more critical during placement. In the current implementation, the final placement descent is executed with a very conservative speed scaling to reduce impact forces and improve insertion accuracy.

After the brick reaches the target placement pose, the system detaches the object from the robot hand in the planning scene and opens the gripper to release the brick. The robot then retreats vertically to a safe height using a Cartesian motion. Once the task has been completed, the active YAML task file is removed. This file-based synchronization prevents the execution node from starting the next action before the perception module and the robot controller have completed the current step.

4.4.3 Collision-Aware Motion Planning and Velocity Scheduling

Velocity control is applied differently across the manipulation phases. Large transit motions such as PREGRASP, LIFT, and PREPLACE use higher velocity scaling factors to improve execution efficiency, while contact-sensitive phases such as DESCENDTOGRASP, DESCENDTOPLACE, and PRESSIN use significantly lower Cartesian velocity and acceleration limits. In the current implementation, free-space motions are typically executed using higher MoveIt velocity scaling parameters, whereas insertion-related Cartesian motions are executed using heavily reduced speed scaling during trajectory time parameterization. This phase-dependent speed scheduling is particularly important for LEGO assembly because fast free-space transport improves task efficiency, while slow final approach and placement improve geometric alignment accuracy and reduce unwanted contact forces between bricks.

The execution node is implemented as a ROS 2 multi-threaded execution pipeline. A multi-threaded ROS 2 executor continuously updates robot joint states, TF transformations, and MoveIt planning-scene information while the main task loop simultaneously waits for incoming YAML assembly tasks. This design ensures that motion planning and trajectory execution always operate on up-to-date robot states and environment geometry. To further improve runtime stability, the MoveGroup interface and planning-scene objects are maintained within a controlled execution scope so that MoveIt resources are released safely during shutdown without leaving active planning objects or controller handles.

4.4.4 Failure Handling

Execution failures are monitored explicitly during runtime. A failure condition is triggered when no valid motion plan can be generated, when the executed pose deviates beyond a predefined tolerance, or when grasping or stud engagement does not succeed. Since each manipulation phase is represented as an independent execution state inside the state-machine framework, failed actions can be retried locally without restarting the entire assembly sequence.

For grasping failures, the state-machine structure can incorporate gripper-state feedback for explicit grasp verification. One possible strategy is to monitor the final gripper width after the closing command has been executed. If the measured finger distance differs significantly from the expected grasp width, the system can infer that the brick was not successfully grasped or slipped during closure. The controller can then retry the grasp using alternative grasp candidates generated during the planning stage.

Similarly, insertion failures can be detected through incomplete Cartesian execution, abnormal contact behavior, or excessive pose deviation during the final placement stage. In such situations, the execution module can perform local recovery behaviors such as repeating the insertion at lower velocity, applying small corrective alignment motions, or returning to a nearby pre-place pose before re-executing the insertion trajectory.

Although the current implementation primarily executes open-loop motions based on estimated object poses, the same state-machine structure provides a clear interface for future extensions. In particular, visual servoing or force-feedback control can be integrated into the `DESCENDTOPLACE` or insertion stages to further compensate for residual pose errors during LEGO stud engagement and improve contact-aware insertion robustness.

5 Evaluation

We evaluate the proposed robotic assembly system in simulation using a custom LEGO Duplo assembly benchmark. The evaluation has three main objectives. First, we analyze whether the benchmark provides a meaningful tiered difficulty progression for robotic assembly tasks. Second, we evaluate the performance of the proposed perception–planning–execution pipeline on multi-step assembly tasks with increasing geometric and structural complexity. Third, we compare the proposed system against a representative vision-language-action (VLA) policy baseline to analyze the advantages and limitations of explicit assembly planning and geometric feasibility reasoning.

5.1 Experimental Setup

Simulation Environment All experiments are conducted in simulation using an adapted LIBERO environment built on top of the MuJoCo physics simulator [25, 42]. The environment models a Franka Emika Panda robot arm with a parallel-jaw gripper, a tabletop workspace, LEGO Duplo bricks, and LEGO-compatible assembly plates. The setup follows the workspace configuration introduced earlier in this thesis, consisting of a dedicated picking area for initially separated parts and an assembly area for target structure construction.

The scene is defined using MuJoCo XML descriptions. LEGO bricks are loaded from STL meshes and scaled to metric units. To separate visual rendering from physical interaction, each brick uses a visible mesh geometry together with simplified collision primitives. The brick body is approximated using box colliders, while studs are modeled using small cylindrical collision geometries, improving contact stability compared with full mesh-based collision handling. The assembly base plate is modeled as a thin rectangular plate with cylindrical studs aligned to the LEGO Duplo grid, enabling stud-based insertion contact and providing a fixed geometric reference for the assembled structure. Contact parameters including friction, `solref`, and `solimp` are tuned to achieve stable quasi-static interactions during grasping, placement, and press-in execution.

Perception Setup The perception pipeline operates on RGB-D observations from two camera viewpoints: a fixed overhead camera for global scene perception and a wrist-mounted camera for local manipulation observations. The overhead camera supports scene-level object detection and pose estimation, while the wrist view improves robustness during close-range manipulation and partially occluded assembly stages.

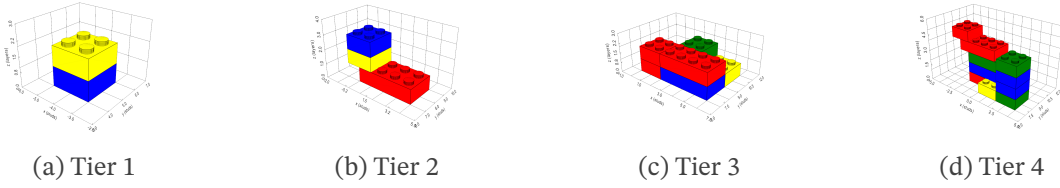


Figure 5.1: Examples of the four task complexity tiers in WorkBenchMark. From left to right: basic vertical stacking (Tier 1), multi-brick sequential stacking (Tier 2), spatial shape layouts (Tier 3), and complex interlocking structures (Tier 4).

For all experiments, CAD models of the LEGO parts are provided beforehand and used for CAD-based 6D pose estimation with FoundationPose. The perception pipeline combines GroundingDINO for open-vocabulary object detection, SAM for instance segmentation, and FoundationPose for RGB-D-based pose estimation. The resulting object poses are transformed into the robot coordinate frame and used by the execution module for grasping and placement.

Task Tiers The benchmark contains four task-complexity tiers, as illustrated in Figure 5.1. Increasing tiers contain more assembly parts, denser spatial occupancy, longer assembly horizons, taller structures, and more constrained insertion operations. Tier 1 contains simple vertical stacking tasks, Tier 2 introduces multi-step sequential assemblies, Tier 3 focuses on spatial layout structures with tighter placement constraints, and Tier 4 contains densely arranged or interlocking structures with more challenging grasping and insertion conditions.

Evaluation Protocol For each tier, $N = 30$ assembly tasks are randomly sampled. Both the proposed system and the VLM/VLA baseline operate under identical workspace conditions and receive the same target structure specifications and initial object configurations. Target structures are represented using structured YAML files that define part identities, assembly poses, and structural relations. During evaluation, the planner derives an executable assembly order, the perception module estimates object poses, and the execution module performs grasping, transportation, alignment, insertion, press-in, and release operations using collision-aware motion planning.

All experiments are conducted on a workstation equipped with an NVIDIA RTX 4000 Ada Generation GPU.

5.2 Baseline Method

Baseline Architecture To provide a representative end-to-end comparison, we implement a vision-language-action (VLA) baseline that combines a vision-language model (VLM) for high-level task decomposition with a pretrained visuomotor policy for low-level manipulation

execution. The baseline is designed to represent a modern language-conditioned robotic manipulation pipeline rather than a manually simplified comparison system.

Starting from the structured target specification, the VLM generates assembly instructions expressed in natural language. In our implementation, we use Gemini 2.5 Flash accessed through an API interface. The prompt contains the final target structure specification represented in YAML format together with task-level assembly requirements and manipulation constraints.

Based on the target specification, the VLM predicts a sequence of manipulation instructions, including the target part, placement relation, relative alignment requirement, insertion direction, and press-in action description. To improve consistency and reduce ambiguous outputs, the prompts are template-guided and include task-specific constraints such as target positions, relative spatial relationships, insertion directions, and vertical press-in requirements.

Policy Execution The generated language instructions are executed by a pretrained visuomotor manipulation policy inspired by the π_0 family of VLA models [5]. The policy operates directly on RGB observations and language instructions without explicit symbolic planning or geometric reasoning. During execution, the policy predicts continuous robot control commands in a closed-loop manner, including end-effector motion updates and gripper open–close actions.

Comparison Conditions Both the proposed system and the VLA baseline operate under the same workspace configuration, target structures, and initial object layouts. The baseline receives the same target assembly specification and visual observations as the proposed method.

Unlike the proposed system, the VLA baseline does not explicitly maintain a structured representation of assembly feasibility during execution. Collision avoidance, insertion alignment, grasp selection, and manipulation ordering are implicitly encoded inside the learned policy representation. This comparison allows us to analyze the role of explicit geometric reasoning and structured assembly planning in contact-rich robotic assembly tasks.

5.3 Evaluation Metrics

We evaluate both systems using task-level metrics related to assembly correctness, execution robustness, and computational efficiency. Table 5.1 summarizes the evaluation metrics used throughout the experiments.

5.3.1 Evaluation Results

Overall Performance Table 5.2 summarizes the quantitative performance of both methods across all task tiers. Overall, the proposed structured perception–planning–execution pipeline consistently outperforms the VLM/VLA-based baseline, and the performance gap becomes

Table 5.1: Task-level evaluation metrics.

Metric	Definition
Assembly Success	If every required brick $v \in V$ is placed at its target pose in the assembly area, attached to the structure, and no longer touched by the robot.
Execution Accuracy	Fraction of brick-to-brick connections whose realised relative pose matches the target. A brick mounted with an offset or not fully pressed down is counted as inaccurate.
Planning Time	Wall-clock time spent on perception, reasoning, and motion planning, excluding physical robot motion.
Manipulation Time	Wall-clock duration of robot motion, from the first command to release of the assembly.
Stability Violation Rate	Fraction of bricks in the assembly area that are not connected to the main structure.

more pronounced as task complexity increases. In Tier 1 and Tier 2, the proposed system achieves success rates of 96.67% and 90.00%, respectively, while maintaining high execution accuracy and low stability violation rates. These results indicate that the planner can reliably derive physically feasible assembly sequences and that the execution module can perform grasping, placement, and stud insertion under relatively simple structural constraints.

As task complexity increases, performance decreases for both methods, but the proposed system remains substantially more robust. In Tier 3 and Tier 4, it still achieves success rates of 70.00% and 66.67%, whereas the VLM/VLA baseline drops to 23.33% and 6.67%. The transition from Tier 2 to Tier 3 introduces a clear increase in structural difficulty: Tier 1 and Tier 2 mainly consist of vertical stacking tasks, while Tier 3 introduces spatial layouts, denser neighboring structures, and narrower insertion clearances. This explains why the performance drop between Tier 2 and Tier 3 is larger than between the earlier tiers.

Runtime The reported planning time also increases with task complexity, rising from 7.13 seconds in Tier 1 to 46.82 seconds in Tier 4 for the proposed system. In our implementation, this runtime includes not only symbolic assembly reasoning but also perception-related computation, including open-vocabulary detection, instance segmentation, CAD-based 6D pose estimation, grasp-feasibility evaluation, collision-aware motion-planning queries, and press-stability verification. As the number of bricks increases, the system must process more candidate parts and perform more independent pose-estimation and feasibility-checking operations, which increases the total computation time. The observed increase should therefore be interpreted as an implementation-level runtime trend rather than an unavoidable theoretical property of the planning method. In particular, repeated perception and feasibility queries could be reduced by caching previously estimated 6D poses, reusing unchanged workspace

Table 5.2: Quantitative comparison across different task tiers. Higher is better for execution accuracy and success rate. Lower is better for planning time, manipulation time and stability violations.

Metric	Tier 1	Tier 2	Tier 3	Tier 4	Overall
Ours (Structured Pipeline)					
Execution Accuracy (%)	93.33	87.22	68.13	66.55	78.81
Planning Time (s)	7.13	13.66	44.78	46.82	28.10
Manipulation Time (s)	8.28	14.08	51.34	53.66	31.84
Stability Violations (%)	3.33	5.00	10.80	25.07	11.05
Success Rate (%)	96.67	90.00	70.00	66.67	80.84
VLM/VLA Baseline					
Execution Accuracy (%)	66.67	48.82	19.34	13.54	37.09
Planning Time (s)	2.18	2.29	3.15	3.66	2.80
Manipulation Time (s)	15.46	48.82	164.45	178.98	101.93
Stability Violations (%)	38.33	55.02	82.97	87.73	68.51
Success Rate (%)	73.33	63.33	23.33	6.67	41.67

observations, and updating only the parts of the voxel state affected by the most recent assembly action.

Manipulation time follows a similar trend because higher-tier tasks contain longer assembly sequences and require more grasping, transportation, placement, insertion, and retreat operations. Since the execution state machine uses largely fixed motion primitives and conservative insertion velocities across all tiers, the increase in manipulation time mainly originates from the larger number of required assembly actions rather than slower individual motions. Compared with the baseline, the proposed system achieves substantially lower manipulation time in complex tiers because explicit geometric reasoning, voxel-based grasp reachability analysis, collision-aware motion planning, and press-stability verification reduce failed grasp attempts, unstable insertions, and repeated recovery motions. The baseline manipulation time increases more sharply because failed insertions often require additional corrective motions, repeated approach attempts, or recovery behavior before the policy can continue.

Baseline Behavior The VLM/VLA baseline shows a much sharper performance degradation as task complexity increases. Although it can complete some simple assembly tasks, its success rate decreases from 73.33% in Tier 1 to only 6.67% in Tier 4, while execution accuracy decreases and stability violations increase substantially. A major limitation is that the baseline does not explicitly model geometric and physical feasibility. Its generated language-level instructions may be semantically plausible but physically infeasible during execution. For example, the baseline may select a reasonable target part or placement relation while failing to account for

grasp clearance, neighboring obstacles, collision-free insertion trajectories, or the downward press required for LEGO stud engagement.

In addition, the VLM/VLA baseline is sensitive to prompt design and instruction quality. Small changes in language phrasing, task decomposition, or spatial descriptions can lead to different manipulation behaviors during execution. Template-guided prompting improves consistency to some extent, but the generated instructions still do not explicitly enforce voxel occupancy constraints, grasp reachability, collision-free insertion, or press stability. This explains the high rate of failed insertions, unstable intermediate assemblies, and collisions in higher-tier tasks.

Failure Cases For the proposed system, most remaining failures in Tier 3 and Tier 4 are related to perception uncertainty and insertion sensitivity. Small errors in CAD-based 6D pose estimation can produce residual misalignment during stud insertion. Since LEGO Duplo assembly requires tight geometric alignment between studs and receiving holes, even small pose deviations may result in incomplete insertion or unstable contact. A second failure mode occurs when the planner selects a sequence that is feasible under the voxel approximation but becomes difficult for the robot because of narrow gripper clearance or accumulated placement error. A third failure mode is press instability in tall or weakly supported structures, where the simplified support-polygon check may not fully capture contact dynamics during insertion.

Overall, the results show that explicit assembly-sequence planning, voxel-based grasp reachability analysis, collision-aware motion planning, and press-stability reasoning provide clear advantages over purely language-driven end-to-end policies for structured robotic assembly. At the same time, the remaining errors suggest that future work should incorporate closed-loop visual servoing or other feedback-based insertion refinement mechanisms during final alignment and press-in execution.

6 Conclusion

This thesis presented a geometry-aware robotic assembly framework for LEGO Duplo assembly tasks. The proposed system integrates structured assembly planning, open-vocabulary perception, and collision-aware execution within a unified robotic pipeline that transforms high-level product specifications into executable robotic manipulation actions. The system was evaluated on a hierarchical LEGO Duplo assembly benchmark designed to analyze robotic assembly performance under progressively increasing geometric and structural complexity.

To generate physically executable assembly sequences, this work introduced an Assembly-by-Disassembly (ABD) planning strategy combined with voxel-based geometric reasoning. The framework incorporates grasp reachability analysis, collision-aware feasibility verification, and press-stability reasoning to handle constrained assembly operations under structural and workspace limitations. In addition, the perception and execution modules enable open-vocabulary object localization, CAD-based 6D pose estimation, and collision-aware robotic manipulation within cluttered assembly environments.

Experimental results demonstrate that the proposed pipeline consistently outperforms the VLM/VLA baseline across all task tiers. In particular, explicit geometric reasoning and physically grounded planning significantly improve execution robustness, insertion stability, and manipulation efficiency in long-horizon assembly tasks. The experiments further show that purely language-driven end-to-end policies frequently generate semantically plausible but physically infeasible assembly actions when geometric constraints, grasp reachability, collision constraints, and structural stability are not explicitly modeled.

The results additionally suggest that hybrid robotic systems combining semantic perception, explicit geometric reasoning, and physically grounded execution currently provide a practical compromise between generalization capability and execution reliability for robotic assembly. While recent VLM/VLA approaches demonstrate strong semantic reasoning and open-world interaction capability, reliable contact-rich robotic assembly still requires explicit modeling of geometry, physical feasibility, and interaction constraints.

At the same time, the experiments reveal several remaining limitations of the current system. In higher-complexity assembly tasks, system performance becomes increasingly sensitive to perception uncertainty and insertion precision. Error accumulation becomes particularly significant in long-horizon assembly sequences, where small inaccuracies in 6D pose estimation or insertion alignment may propagate across multiple manipulation stages and eventually lead to insertion failure or unstable contact behavior. In addition, the current press-stability

analysis still relies on simplified geometric approximations and does not yet model more complex physical interaction effects such as force propagation, contact dynamics, or deformable compliance during insertion.

Future work will focus on incorporating closed-loop visual servoing and feedback-based refinement during the final alignment and insertion stages. Such feedback mechanisms could compensate for residual pose-estimation errors and further improve insertion robustness in highly constrained assembly scenarios. Future planning methods may also incorporate more accurate press-stability analysis, force-aware insertion reasoning, and richer contact modeling. Additional future directions include transferring the framework from simulation to real robotic platforms and enabling dynamic replanning under uncertain or partially observable environments. These directions aim to improve both geometric robustness and adaptive interaction capability during contact-rich robotic assembly.

Beyond LEGO assembly, the proposed framework also demonstrates the broader potential of combining structured planning with open-vocabulary perception for general robotic manipulation. Since the perception pipeline is not restricted to predefined object categories, the system can in principle generalize to previously unseen objects and manipulation targets. This capability is particularly relevant for future household-service robots, tabletop manipulation systems, and flexible embodied-AI environments, where robots must operate in open and dynamically changing scenes while interacting with diverse objects under physical constraints.

Overall, this work demonstrates the importance of combining geometric reasoning, physically grounded planning, and collision-aware execution for reliable long-horizon robotic assembly in open and contact-rich environments.

List of Acronyms

PDF Probability Density Function

RV Random Variable

List of Symbols

General

$(\cdot)$	arbitrary (scalar) variable
$(\bullet)$	arbitrary vector variable
$\approx$	approximately equal
$\langle \cdot, \cdot, \cdot \rangle$	tuple of multiple variables
$\mathbb{R}$	set of real numbers
$\mathbb{E}[X]$	expected value of a random variable (RV) X
$\mathbb{P}[X = x]$	probability that an RV X takes on the value x
$\frac{\partial f(\cdot)}{\partial x}$	partial derivative of f with respect to x
$\left. \frac{\partial f(\cdot)}{\partial x} \right _a$	partial derivative of f with respect to x evaluated at a
$\arg \min_a f(a)$	optimal value of a minimizing the expression $f(a)$
$\arg \max_a f(a)$	optimal value of a maximizing the expression $f(a)$
π	mathematical constant, $\pi \approx 3.142$
$\mathcal{N}(\mu, \sigma)$	Gaussian normal distribution with mean μ and standard deviation σ

Markov Decision Process

t	time step index
$\mathcal{S}$	set of all states
$\mathcal{A}$	set of all actions
$\mathcal{R}$	set of all possible rewards
s, s^t	state at time t , realization of RV S^t
s', s^{t+1}	next state at time $t + 1$, realization of RV S^{t+1}
a, a^t	action at time t , realization of RV A^t
a', a^{t+1}	next action at time $t + 1$, realization of RV A^{t+1}
r, r^{t+1}	reward at time $t + 1$, realization of RV R^{t+1}
S^t	state at time t
A^t	action at time t
R^t	reward at time t
$p(s', r \mid s, a)$	dynamics function specifying a conditional probability density function (PDF) for the next state s' and reward r given the current state s and action a

List of Figures

1.1	Example robotic assembly task used in this work. Separated LEGO Duplo bricks are initially placed in the pick area (left), and the robot constructs the target structure in the assembly area (right).	2
1.2	Robotic assembly workspace used in this work, consisting of a Franka Emika Panda robot arm, an RGB-D camera, a pick area containing separated LEGO Duplo bricks, and an assembly area for target structure construction.	3
2.1	Examples of open-vocabulary perception in cluttered environments. The right image is adapted from [35].	8
2.2	Overview of FoundationPose and example 6D pose estimation results. Left: unified model-based and model-free pose estimation framework. Right: example predicted object pose with aligned coordinate axes. Adapted from [45].	11
4.1	System data flow from a structured YAML assembly goal to sequence planning, open-vocabulary perception, and collision-aware robot execution.	23
4.2	Example of ABD reasoning with inferred disassembly order and candidate grasp-point pairs. The numbered blocks indicate the inferred removal sequence, while the black markers represent candidate grasp-point pairs evaluated during planning.	28
4.3	Examples of grasp reachability evaluation under voxel-based gripper occupancy constraints. The dashed regions visualize the approximated gripper occupancy area represented as a 2×2 voxel region aligned with the LEGO stud grid. A grasp candidate is considered feasible only if the occupancy region remains collision-free under the current scene configuration. Left: feasible grasp candidate with sufficient free clearance. Right: infeasible grasp candidate where the occupancy region intersects with neighboring bricks. The numbers indicate the candidate evaluation order during ABD search.	30
4.4	Perception pipeline for active part localisation in a LEGO Duplo workspace. From left to right: RGB-D workspace input, GroundingDINO box, SAM instance mask, and FoundationPose 6D pose.	31
4.5	Execution state machine for one pick-and-place assembly step. The controller moves from free-space transport to slow insertion, release, and retreat before the next planned part is executed.	34

- 5.1 Examples of the four task complexity tiers in WorkBenchMark. From left to right: basic vertical stacking (Tier 1), multi-brick sequential stacking (Tier 2), spatial shape layouts (Tier 3), and complex interlocking structures (Tier 4). . . 38

List of Tables

5.1	Task-level evaluation metrics.	40
5.2	Quantitative comparison across different task tiers. Higher is better for execution accuracy and success rate. Lower is better for planning time, manipulation time and stability violations.	41

List of Algorithms

1	ABD reasoning with grasp reachability and press-stability checks	26
---	--	----

List of References

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, *Do as i can, not as i say: Grounding language in robotic affordances*, 2022. arXiv: 2204.01691 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2204.01691>.
- [2] J. Aleotti and S. Caselli, “Efficient planning of disassembly sequences in physics-based animation,” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 87–92, 2009. DOI: 10.1109/IROS.2009.5354401.
- [3] A. Barghi, N. Pasiar, M. T. Masouleh, and A. Kalhor, “From bricks to bots: Automated collision-aware sequence planning for LEGO reconstruction with a two-finger gripper,” in *2024 10th International Conference on Control, Instrumentation and Automation (ICCIA)*, 2024, pp. 1–6. DOI: 10.1109/ICCIA65044.2024.10768173.
- [4] D. Beßler, M. Pomarlan, A. Akbari, Muhayyuddin, M. Diab, J. Rosell, J. Bateman, and M. Beetz, “Assembly planning in cluttered environments through heterogeneous reasoning,” in *KI 2018: Advances in Artificial Intelligence*, Springer, 2018, pp. 290–304. DOI: 10.1007/978-3-030-00111-7_25.
- [5] K. Black et al., π_0 : *A vision-language-action flow model for general robot control*, 2024. arXiv: 2410.24164 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2410.24164>.
- [6] J. Bohren and S. Cousins, “SMACH: A task-level architecture for rapidly creating complex robot behavior,” in *IEEE International Conference on Robotics and Automation Workshop on Modular Software for Robotics and Automation*, 2011.
- [7] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2212.06817>.
- [8] A. Brohan et al., *RT-2: Vision-language-action models transfer web knowledge to robotic control*, 2023. arXiv: 2307.15818 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2307.15818>.

-
- [9] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986. DOI: 10.1109/JRA.1986.1087032.
 - [10] S. Chitta, I. A. Sucan, and S. Cousins, “MoveIt!” *IEEE Robotics & Automation Magazine*, vol. 19, no. 1, pp. 18–19, 2012. DOI: 10.1109/MRA.2011.2181749.
 - [11] D. Coleman, I. A. Sucan, S. Chitta, and N. Correll, “Reducing the barrier to entry of complex robotic software: A MoveIt! case study,” *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014.
 - [12] R. E. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971. DOI: 10.1016/0004-3702(71)90010-5.
 - [13] N. Funk, S. Menzenbach, G. Chavatzaki, and J. Peters, “Graph-based reinforcement learning meets mixed integer programs: An application to 3d robot assembly discovery,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022. arXiv: 2203.04120. [Online]. Available: <https://arxiv.org/abs/2203.04120>.
 - [14] D. Halperin, J.-C. Latombe, and R. H. Wilson, “A general framework for assembly planning: The motion space approach,” *Algorithmica*, vol. 26, no. 3–4, pp. 577–601, 2000. DOI: 10.1007/s004539910024.
 - [15] M. Heo, Y. Lee, D. Lee, and J. J. Lim, *Furniturebench: Reproducible real-world benchmark for long-horizon complex manipulation*, 2023. arXiv: 2305.12821 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2305.12821>.
 - [16] S. Hinterstoisser, V. Lepetit, S. Ilic, S. Holzer, G. Bradski, K. Konolige, and N. Navab, “Model based training, detection and pose estimation of texture-less 3d objects in heavily cluttered scenes,” in *Asian Conference on Computer Vision (ACCV)*, Springer, 2012, pp. 548–562.
 - [17] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, *Inner monologue: Embodied reasoning through planning with language models*, 2022. arXiv: 2207.05608 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2207.05608>.
 - [18] S. Hutchinson, G. D. Hager, and P. I. Corke, “A tutorial on visual servo control,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 5, pp. 651–670, 1996. DOI: 10.1109/70.538972.
 - [19] M. J. Kim et al., *OpenVLA: An open-source vision-language-action model*, 2024. arXiv: 2406.09246 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2406.09246>.
 - [20] S.-K. Kim and M. Likhachev, “Parts assembly planning under uncertainty with simulation-aided physical reasoning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 4074–4081. DOI: 10.1109/ICRA.2017.7989463.

-
- [21] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, “Segment anything,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4015–4026.
 - [22] K. Kitz and U. Thomas, “Neural dynamic assembly sequence planning,” in *IEEE International Conference on Automation Science and Engineering (CASE)*, 2021, pp. 2063–2068. DOI: 10.1109/CASE49439.2021.9551570.
 - [23] D. Le, J. Cortés, and T. Siméon, “A path planning approach to (dis)assembly sequencing,” in *IEEE International Conference on Automation Science and Engineering (CASE)*, 2009, pp. 286–291. DOI: 10.1109/COASE.2009.5234177.
 - [24] Y. Lee, E. S. Hu, Z. Yang, A. Yin, and J. J. Lim, *IKEA furniture assembly environment for long-horizon complex manipulation tasks*, 2019. arXiv: 1911.07246 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/1911.07246>.
 - [25] B. Liu, Y. Liu, X. Geng, F. Bu, C. Chi, Y. Yang, Y. Zhu, P. Stone, Q. Liu, and M. Tomizuka, *LIBERO: Benchmarking knowledge transfer for lifelong robot learning*, 2023. arXiv: 2306.03310 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2306.03310>.
 - [26] R. Liu, K. Deng, Z. Wang, and C. Liu, *Stablelego: Stability analysis of block stacking assembly*, 2024. arXiv: 2402.10711 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2402.10711>.
 - [27] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision (ECCV)*, 2024. arXiv: 2303.05499. [Online]. Available: <https://arxiv.org/abs/2303.05499>.
 - [28] J. Luo, C. Xu, and S. Levine, “Learning force-based manipulation for precise robotic assembly,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
 - [29] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022. DOI: 10.1126/scirobotics.abm6074.
 - [30] L. S. H. de Mello and A. C. Sanderson, “AND/OR graph representation of assembly plans,” *IEEE Transactions on Robotics and Automation*, vol. 6, no. 2, pp. 188–199, 1990. DOI: 10.1109/70.54702.
 - [31] L. S. H. de Mello and A. C. Sanderson, “A correct and complete algorithm for the generation of mechanical assembly sequences,” *IEEE Transactions on Robotics and Automation*, vol. 7, no. 2, pp. 228–240, 1991. DOI: 10.1109/70.75905.
 - [32] L. Nägele, A. Hoffmann, A. Schierl, and W. Reif, “Legobot: Automated planning for coordinated multi-robot assembly of LEGO structures,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 9088–9095. DOI: 10.1109/IROS45743.2020.9341428.

-
- [33] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
 - [34] S. Rakshit and S. Akella, “The influence of motion paths and assembly sequences on the stability of assemblies,” *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 615–626, 2015. DOI: 10.1109/TASE.2014.2327011.
 - [35] T. Ren, S. Liu, A. Zeng, J. Lin, K. Li, H. Cao, J. Chen, X. Huang, Y. Chen, F. Yan, Z. Zeng, H. Zhang, F. Li, J. Yang, H. Li, Q. Jiang, and L. Zhang, *Grounded SAM: Assembling open-world models for diverse visual tasks*, 2024. arXiv: 2401.14159 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2401.14159>.
 - [36] A. C. Sanderson, L. S. H. de Mello, and H. Zhang, “Assembly sequence planning,” *AI Magazine*, vol. 11, no. 1, pp. 62–81, 1990. DOI: 10.1609/aimag.v11i1.824.
 - [37] I. A. Sutan, M. Moll, and L. E. Kavraki, “The open motion planning library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012. DOI: 10.1109/MRA.2012.2205651.
 - [38] F. Sun, R. Chen, T. Ji, Y. Luo, H. Zhou, and H. Liu, *A comprehensive survey on embodied intelligence: Advancements, challenges, and future perspectives*, 2024. DOI: 10.26599/AIR.2024.9150042.
 - [39] S. Sundaram, I. Remmler, and N. M. Amato, “Disassembly sequencing using a motion planning approach,” in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, 2001, pp. 1475–1480. DOI: 10.1109/ROBOT.2001.932818.
 - [40] Y. Tian, K. D. D. Willis, B. A. Omari, J. Luo, P. Ma, Y. Li, F. Javid, E. Gu, J. Jacob, S. Sueda, H. Li, S. Chitta, and W. Matusik, *ASAP: Automated sequence planning for complex robotic assembly with physical feasibility*, Accepted to ICRA 2024, 2024. arXiv: 2309.16909 [cs.RO]. [Online]. Available: <https://arxiv.org/abs/2309.16909>.
 - [41] Y. Tian, J. Xu, Y. Li, J. Luo, S. Sueda, H. Li, K. D. D. Willis, and W. Matusik, “Assemble them all: Physics-based planning for generalizable assembly by disassembly,” *ACM Transactions on Graphics*, vol. 41, no. 6, pp. 1–11, 2022. DOI: 10.1145/3550454.355525.
 - [42] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033. DOI: 10.1109/IRoS.2012.6386109.
 - [43] C. Wang, D. Xu, Y. Zhu, R. Martín-Martín, C. Lu, L. Fei-Fei, and S. Savarese, “Densefusion: 6d object pose estimation by iterative dense fusion,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3343–3352.
 - [44] K. Watanabe and S. Inada, “Search algorithm of the assembly sequence of products by using past learning results,” *International Journal of Production Economics*, vol. 226, p. 107615, 2020. DOI: 10.1016/j.ijpe.2019.107615.

-
- [45] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “Foundationpose: Unified 6d pose estimation and tracking of novel objects,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv: 2312.08344. [Online]. Available: <https://arxiv.org/abs/2312.08344>.
 - [46] R. H. Wilson and J.-C. Latombe, “Geometric reasoning about mechanical assembly,” *Artificial Intelligence*, vol. 71, no. 2, pp. 371–396, 1994. DOI: 10.1016/0004-3702(94)90048-5.
 - [47] J. D. Winter, E. Makrini, G. V. de Perre, A. Nowé, T. Verstraeten, and B. Vanderborght, “Autonomous assembly planning of demonstrated skills with reinforcement learning in simulation,” *Autonomous Robots*, vol. 45, no. 8, pp. 1097–1110, 2021. DOI: 10.1007/s10514-021-10003-w.
 - [48] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, *PoseCNN: A convolutional neural network for 6d object pose estimation in cluttered scenes*, 2017. arXiv: 1711.00199 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1711.00199>.
 - [49] Z. Xu, K. Wu, J. Wen, J. Li, N. Liu, Z. Che, and J. Tang, *A survey on robotics with foundation models: Toward embodied AI*, 2024. arXiv: 2402.02385 [cs.RO]. [Online]. Available: <https://arxiv.org/abs/2402.02385>.
 - [50] M. Yu, Z. Ling, S. Li, H. Geng, T. Chen, J. Deng, N. Jiang, A. Markham, and X. Huang, “Roboassembly: Learning generalizable furniture assembly policy in a novel multi-robot contact-rich simulation environment,” *arXiv preprint arXiv:2112.10143*, 2021. [Online]. Available: <https://arxiv.org/abs/2112.10143>.
 - [51] X. Zhang, R. Belfer, P. G. Kry, and E. Vouga, “C-space tunnel discovery for puzzle path planning,” *ACM Transactions on Graphics*, vol. 39, no. 4, 104:1–104:14, 2020. DOI: 10.1145/3386569.3392468.